

http://algs4.cs.princeton.edu

4.1 UNDIRECTED GRAPHS

- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

4.1 UNDIRECTED GRAPHS

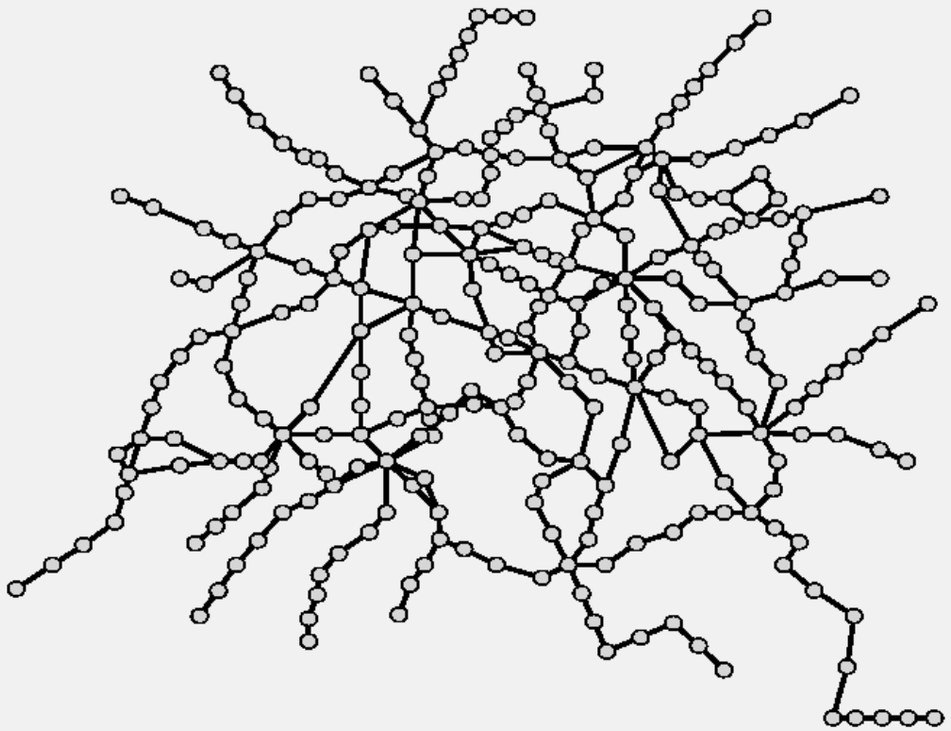
- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

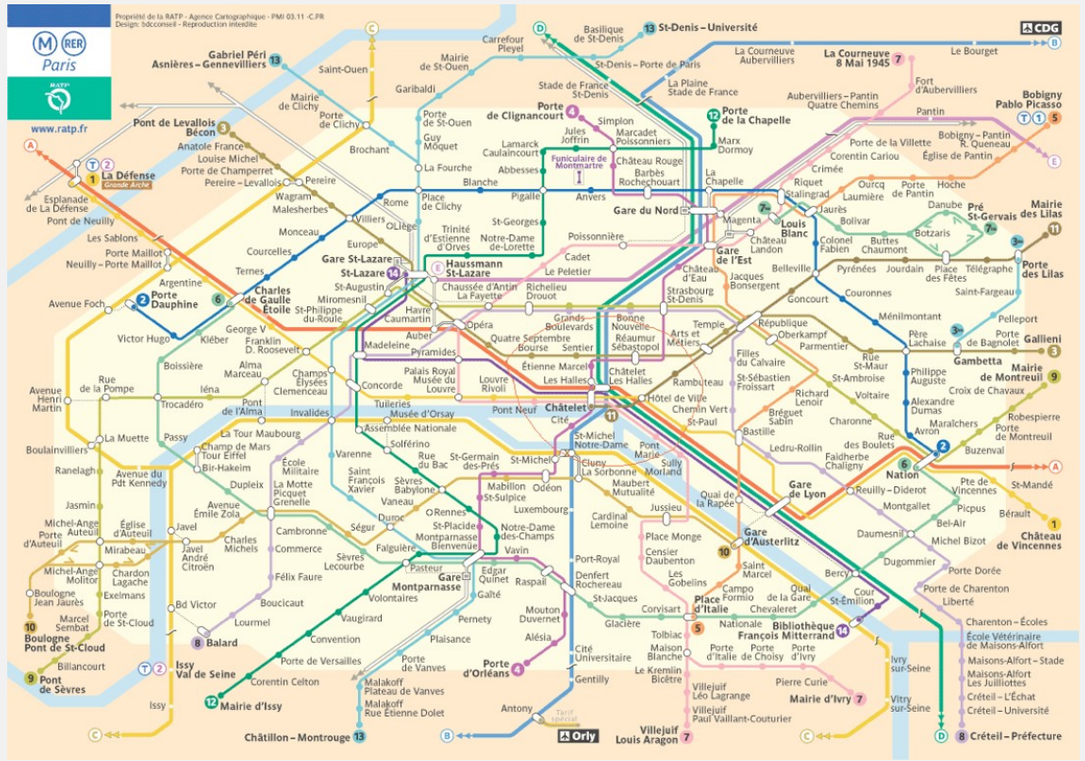
Undirected graphs

Graph. Set of vertices connected pairwise by edges.

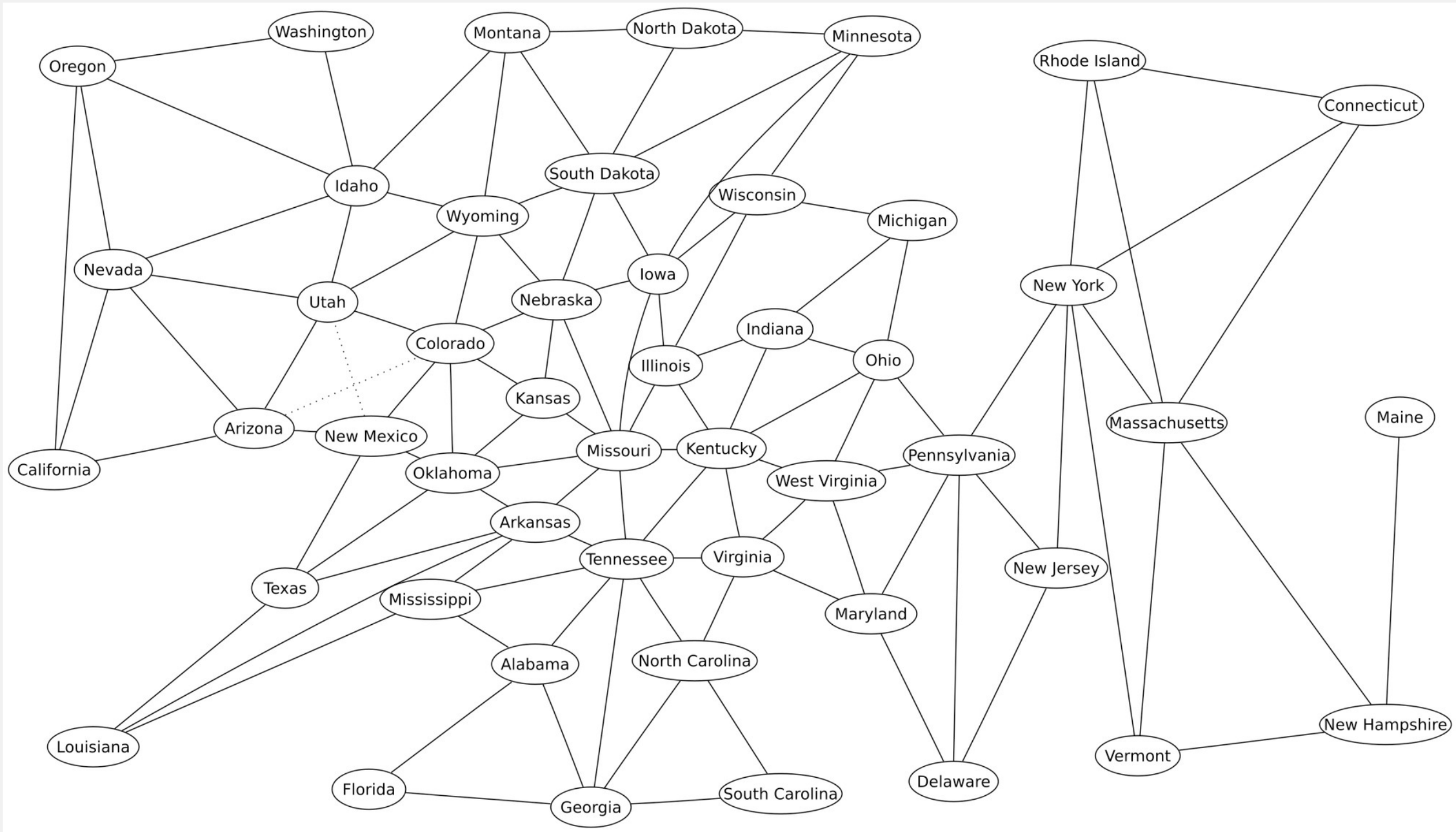
Why study graph algorithms?

- Thousands of practical applications.
- Hundreds of graph algorithms known.
- Interesting and broadly useful abstraction.
- Challenging branch of computer science and discrete math.

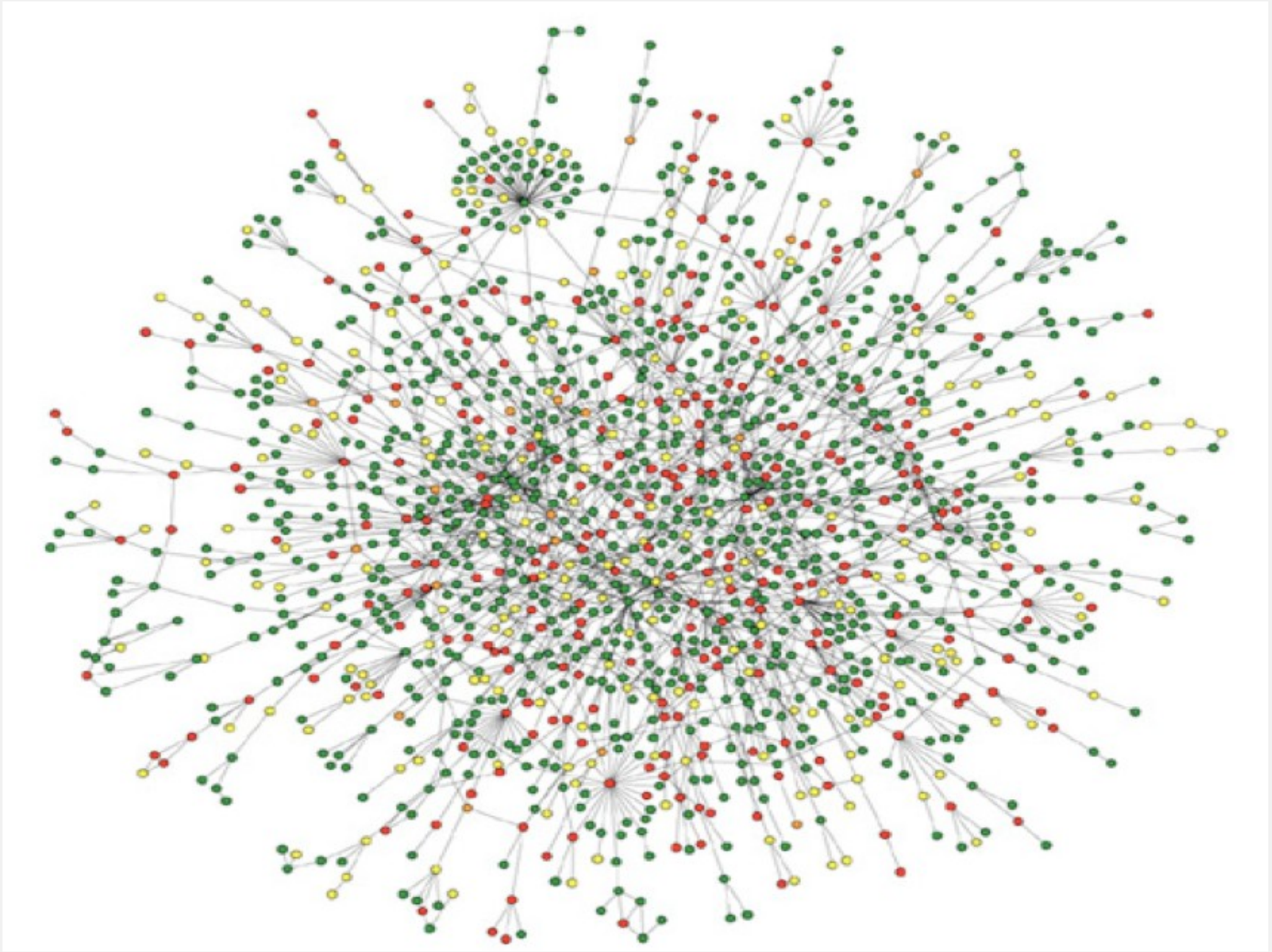




Border graph of 48 contiguous United States

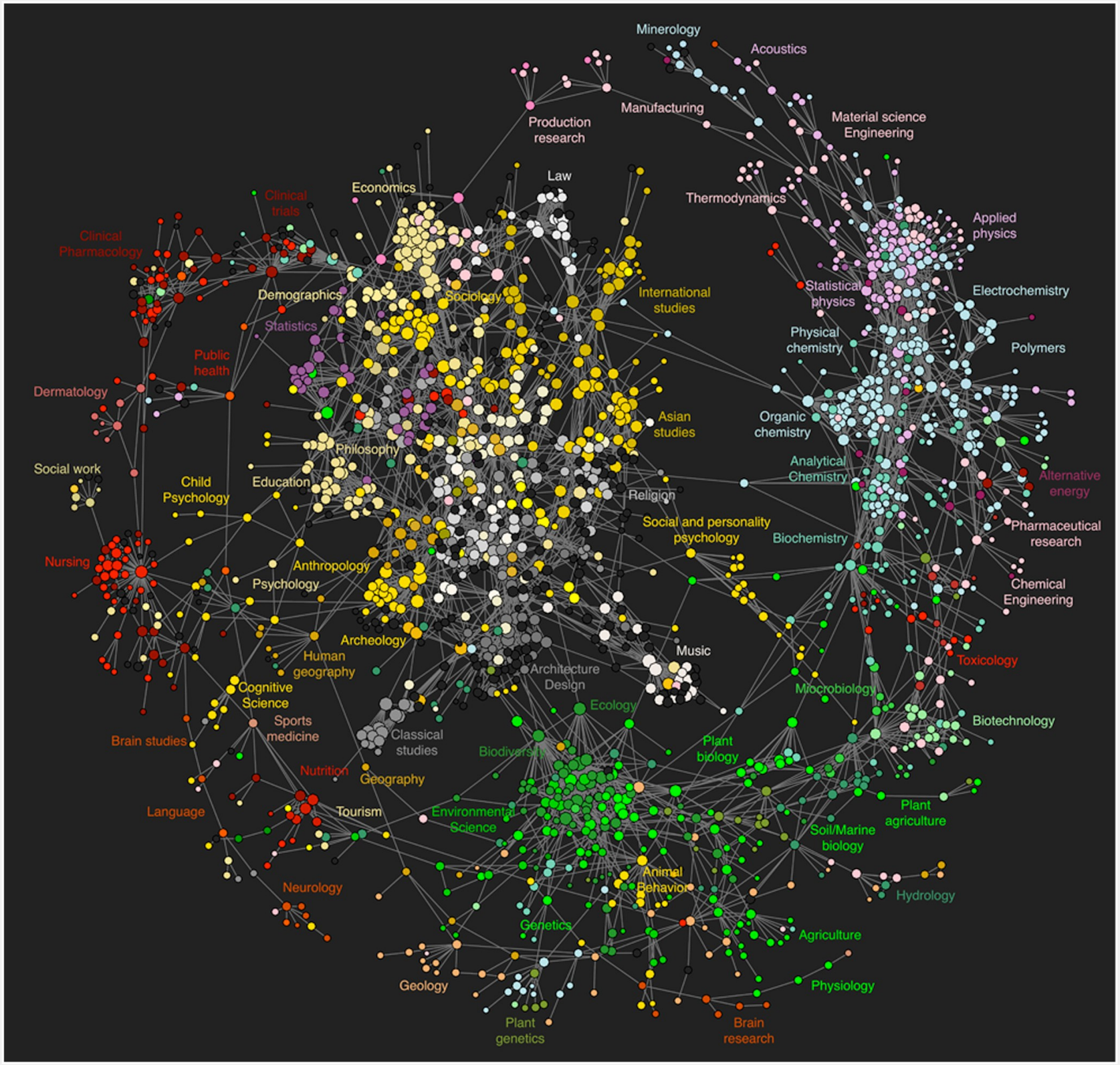


Protein-protein interaction network



Reference: Jeong et al, Nature Review | Genetics

Map of science clickstreams



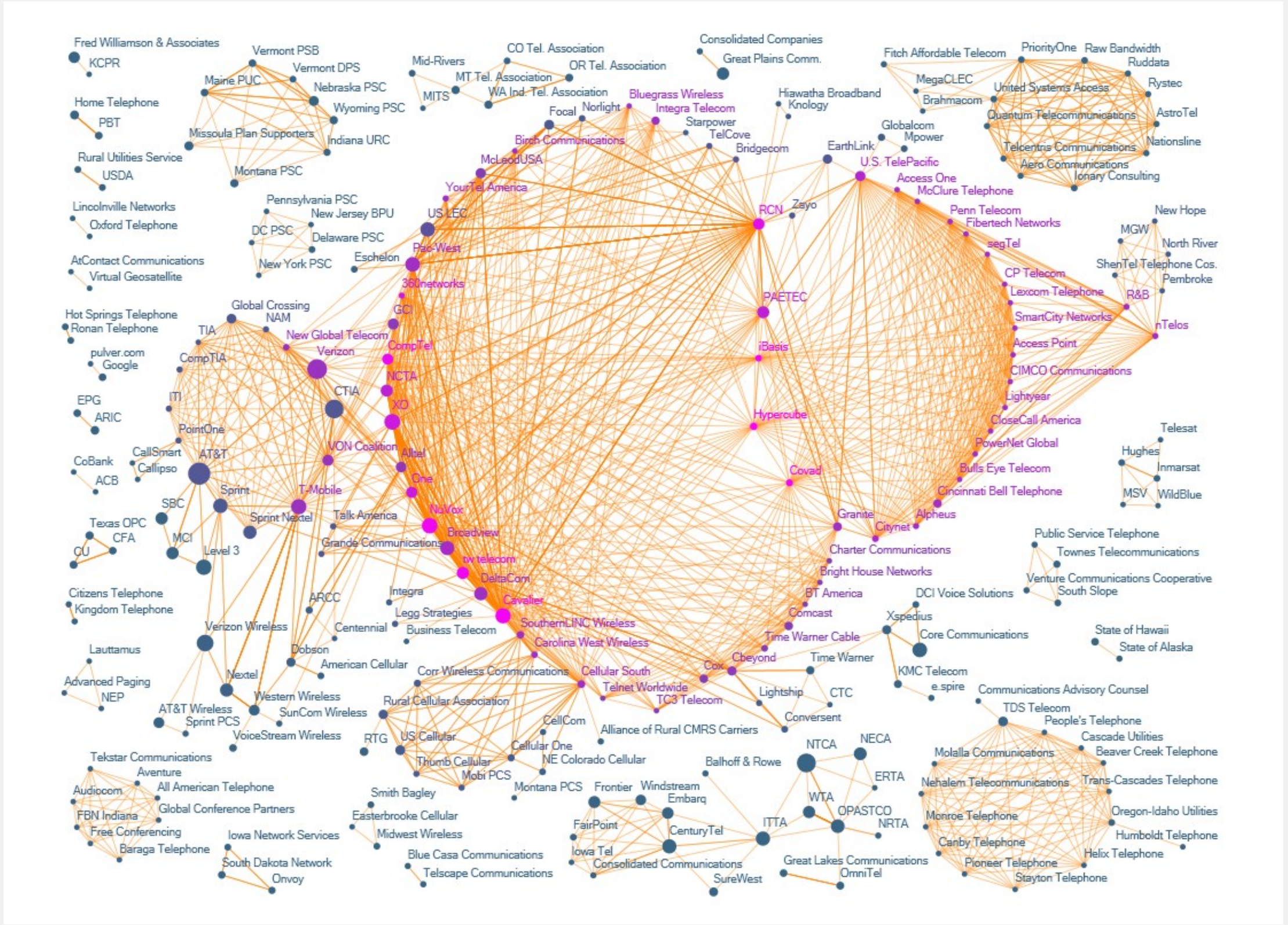
http://www.plosone.org/article/info:doi/10.1371/journal.pone.0004803

10 million Facebook friends



"Visualizing Friendships" by Paul Butler

The evolution of FCC lobbying coalitions



"The Evolution of FCC Lobbying Coalitions" by Pierre de Vries in JoSS Visualization Symposium 2010

Framingham heart study

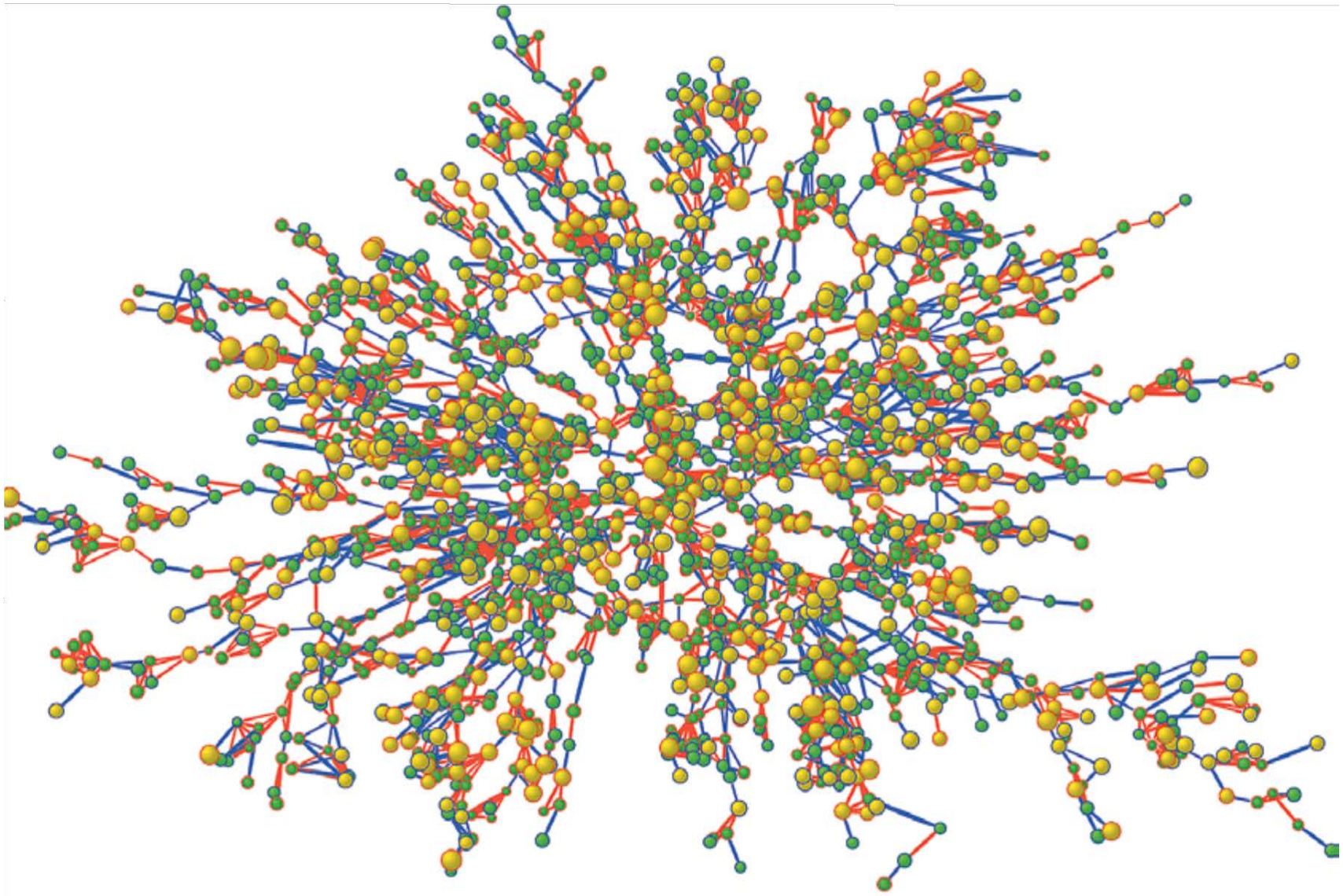
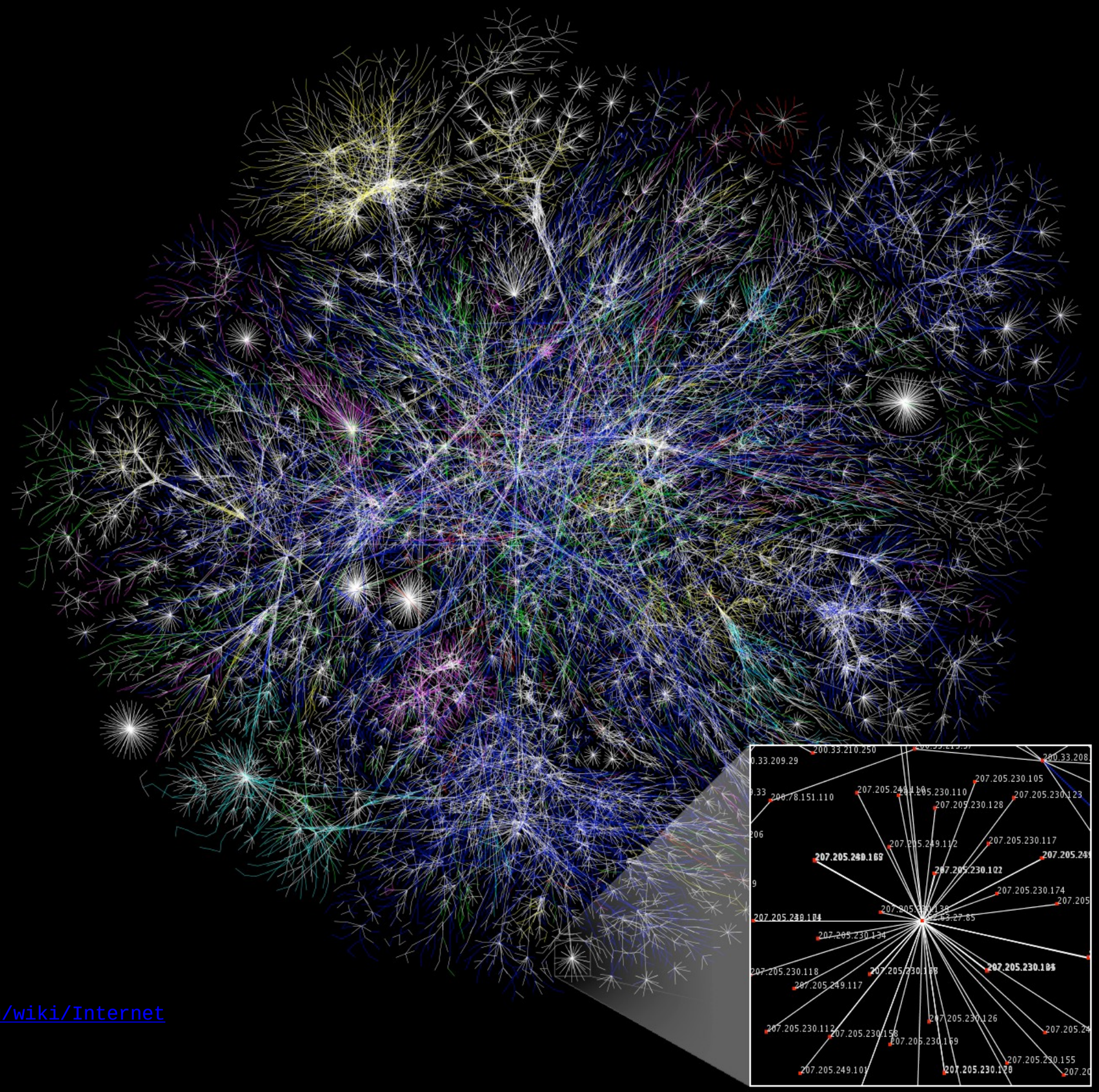


Figure 1. Largest Connected Subcomponent of the Social Network in the Framingham Heart Study in the Year 2000.
Each circle (node) represents one person in the data set. There are 2200 persons in this subcomponent of the social network. Circles with red borders denote women, and circles with blue borders denote men. The size of each circle is proportional to the person's body-mass index. The interior color of the circles indicates the person's obesity status: yellow denotes an obese person (body-mass index, ≥30) and green denotes a nonobese person. The colors of the ties between the nodes indicate the relationship between them: purple denotes a friendship or marital tie and orange denotes a familial tie.

"The Spread of Obesity in a Large Social Network over 32 Years" by Christakis and Fowler in New England Journal of Medicine, 2007

The Internet as mapped by the Opte Project



http://en.wikipedia.org/wiki/Internet

Graph applications

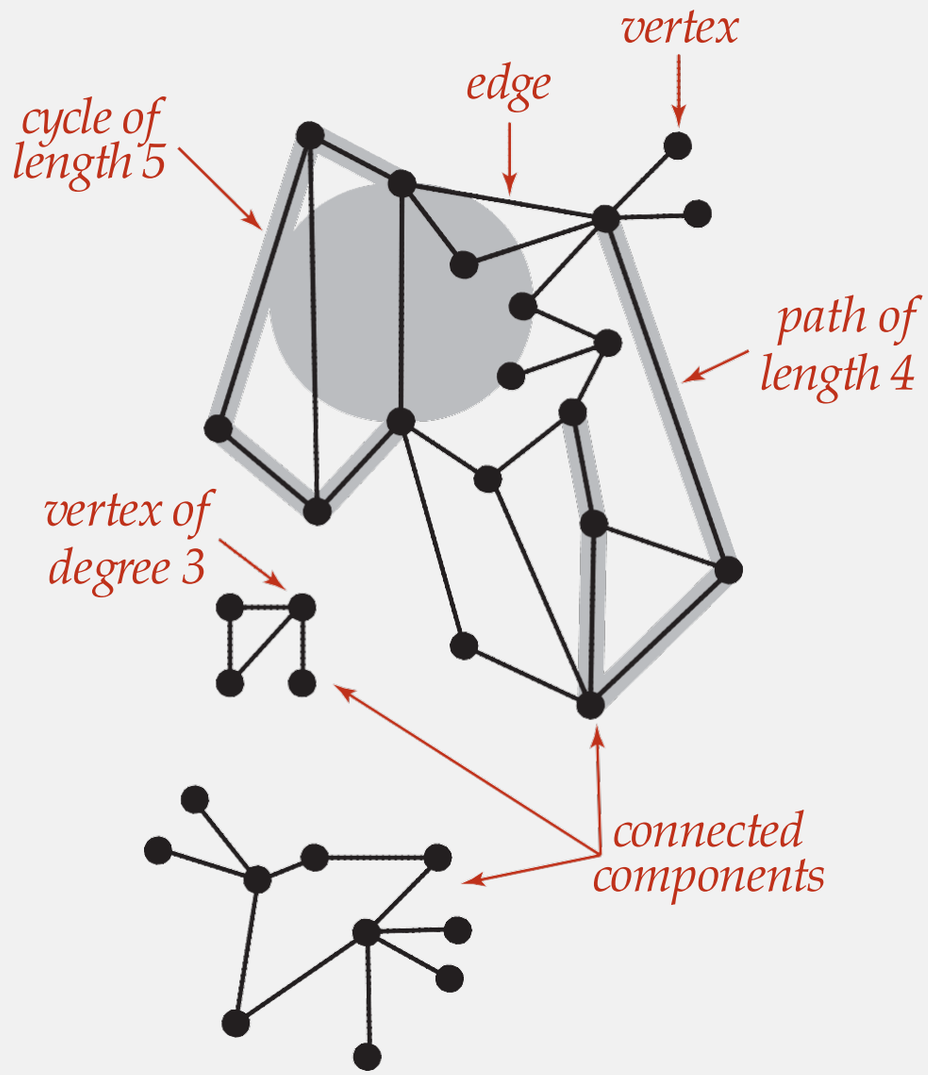
graph	vertex	edge
communication	telephone, computer	fiber optic cable
circuit	gate, register, processor	wire
mechanical	joint	rod, beam, spring
financial	stock, currency	transactions
transportation	intersection	street
internet	class C network	connection
game	board position	legal move
social relationship	person	friendship
neural network	neuron	synapse
protein network	protein	protein-protein interaction
molecule	atom	bond

Graph terminology

Path. Sequence of vertices connected by edges.

Cycle. Path whose first and last vertices are the same.

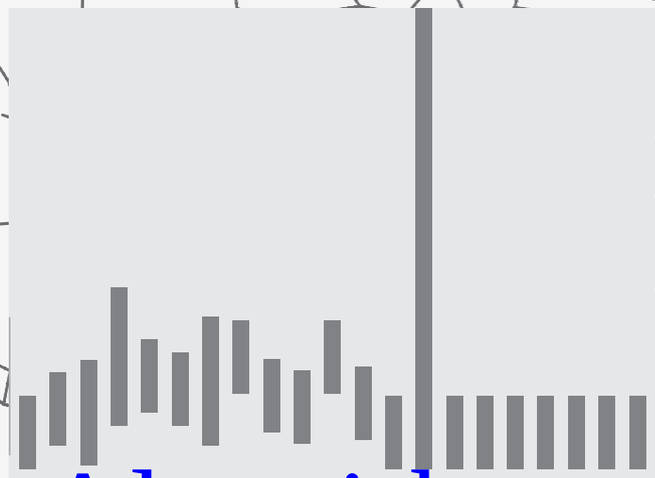
Two vertices are connected if there is a path between them.



Some graph-processing problems

problem	description
s-t path	*Is there a path between s and t ?*
shortest s-t path	*What is the shortest path between s and t ?*
cycle	*Is there a cycle in the graph ?*
Euler cycle	*Is there a cycle that uses each edge exactly once ?*
Hamilton cycle	*Is there a cycle that uses each vertex exactly once ?*
connectivity	*Is there a way to connect all of the vertices ?*
biconnectivity	*Is there a vertex whose removal disconnects the graph ?*
planarity	*Can the graph be drawn in the plane with no crossing edges ?*
graph isomorphism	*Do two adjacency lists represent the same graph ?*

Challenge. Which graph problems are easy? difficult? intractable?



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

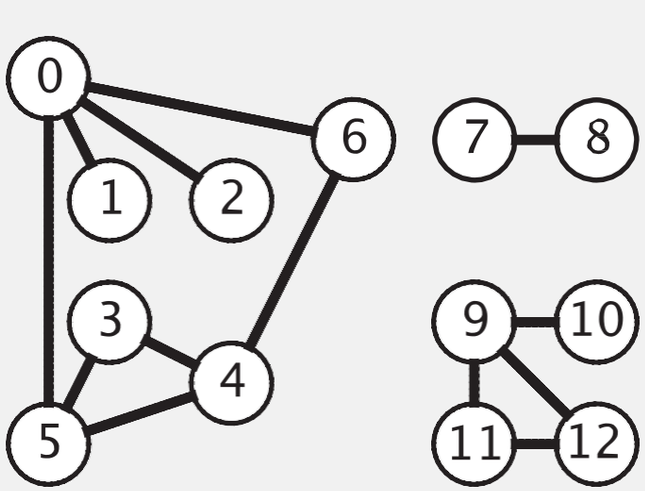
http://algs4.cs.princeton.edu

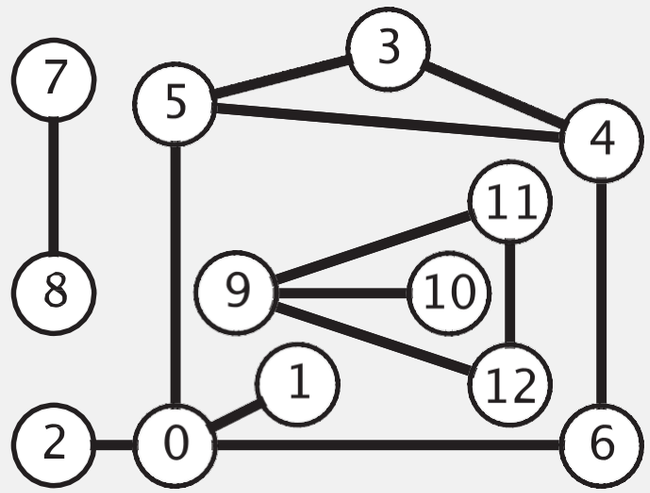
4.1 UNDIRECTED GRAPHS

- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

Paraqitja e grafeve

Vizatimi i grafeve. Rritë intuitën për strukturën e grafit.





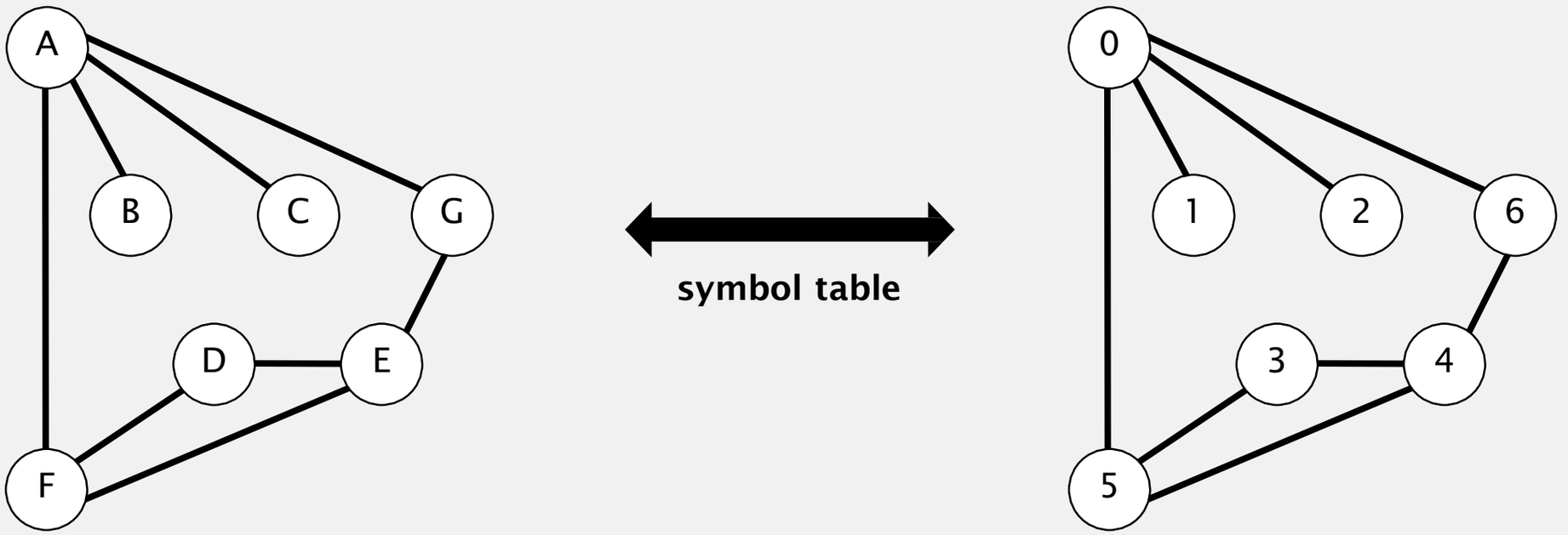
dy vizatime të të njejtit graf

Vërejtje. Intuita mund të dërgon në drejtim të gabuar

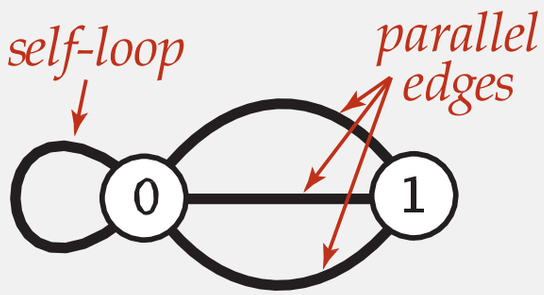
Paraqitja e grafeve

Paraqitja e kulmeve.

- Ligjërata: përdorë integer prej 0 deri në $V - 1$.
- Aplikacioni: konverto në mes të emrit dhe integer-it me anë të symbol table.



Anomalitë.



Graph API

public class Graph		
	Graph(int V)	*create an empty graph with V vertices*
	Graph(In in)	*create a graph from input stream*
void	addEdge(int v, int w)	*add an edge v-w*
Iterable<Integer>	adj(int v)	*vertices adjacent to v*
int	V()	*number of vertices*
int	E()	*number of edges*

```
In in = new In(args[0]);
Graph G = new Graph(in);

for (int v = 0; v < G.V(); v++)
   for (int w : G.adj(v))
      StdOut.println(v + "-" + w);
```

read graph from input stream

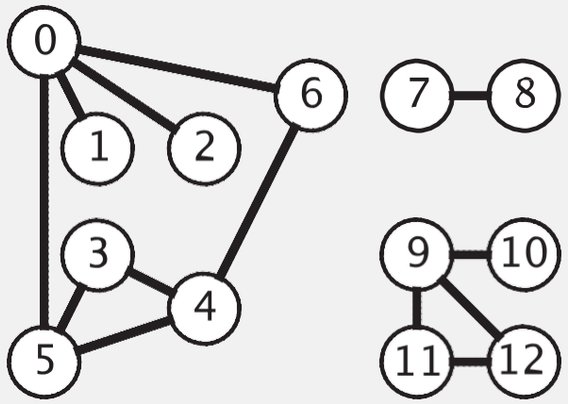
print out each edge (twice)

Graph API: shembull

Graph input format.

tinyG.txt

```
V → 13
E → 13
0  5
4  3
0  1
9  12
6  4
5  4
0  2
11  12
9  10
0  6
7  8
9  11
5  3
```




```
% java Test tinyG.txt
0-6

0-2
0-1
0-5
1-0
2-0
3-5
3-4
⋮
12-11
12-9
```

```
In in = new In(args[0]);
Graph G = new Graph(in);

for (int v = 0; v < G.V(); v++)
   for (int w : G.adj(v))
      StdOut.println(v + "-" + w);
```

read graph from input stream

print out each edge (twice)

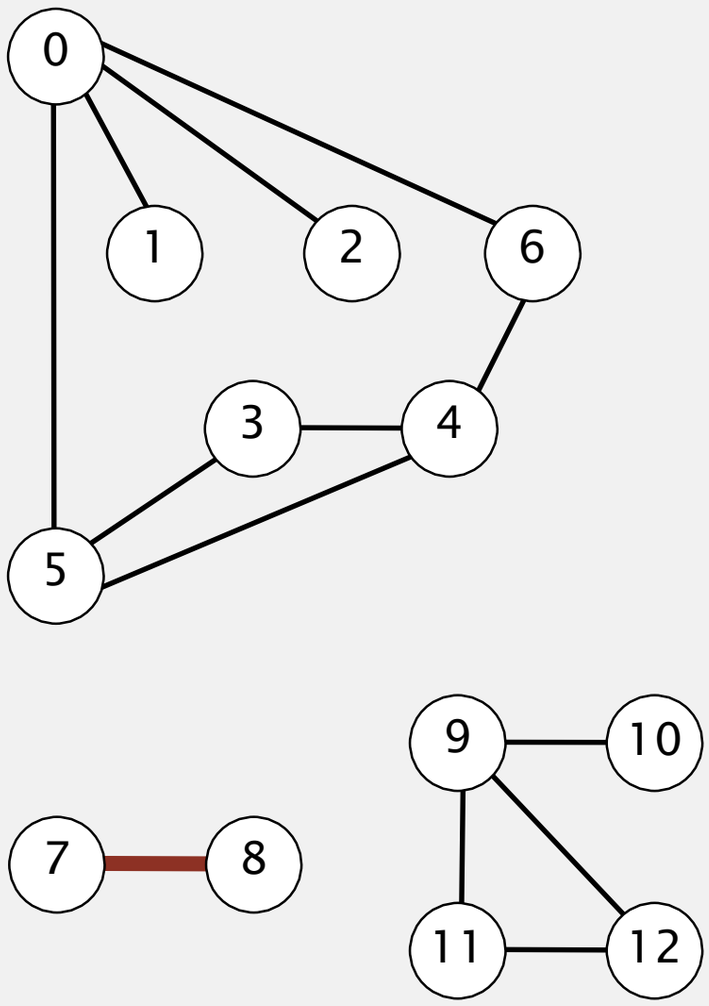
kod për graph-processing

public class Graph		
	Graph(int V)	*create an empty graph with V vertices*
	Graph(In in)	*create a graph from input stream*
void	addEdge(int v, int w)	*add an edge v-w*
Iterable<Integer>	adj(int v)	*vertices adjacent to v*
int	V()	*number of vertices*
int	E()	*number of edges*

```
// degree of vertex v in graph G
public static int degree(Graph  G, int v)
{
    int degree = 0;
    for (int w : G.adj(v))
       degree++;
    return degree;
}
```

Paraqitja e Set-of-edges graph

Mirëmbajtja e list of the edges (linked list or array).



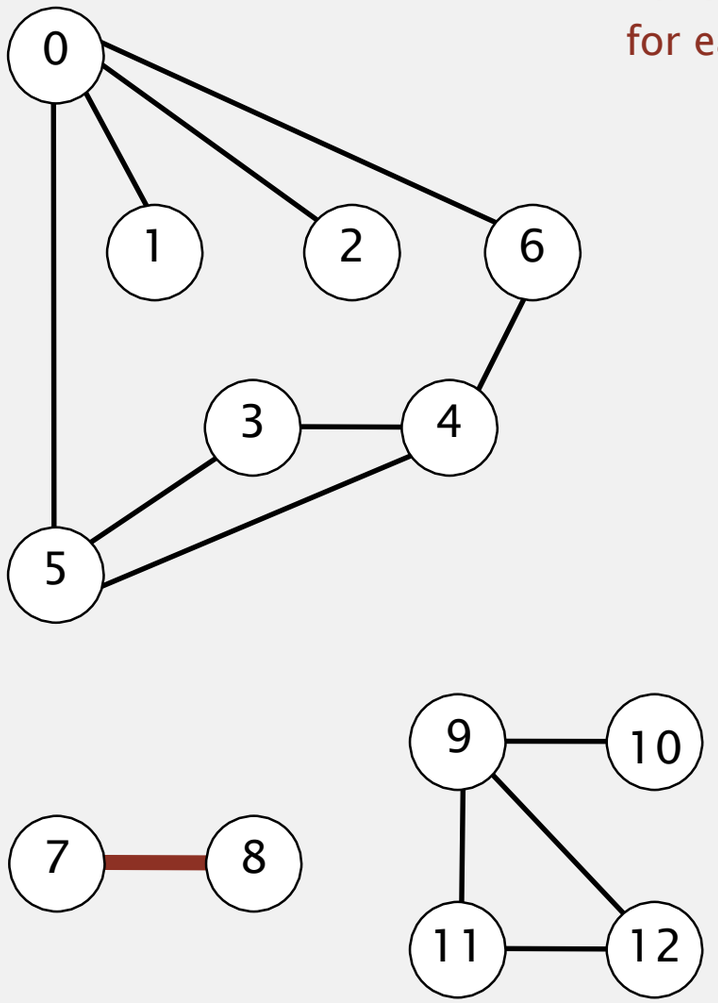
0	1
0	2
0	5
0	6
3	4
3	5
4	5
4	6
7	8
9	10
9	11
9	12
11	12

Q. How long to iterate over vertices adjacent to v ?

Paraqitja me Adjacency-matrix graph

Mirëmbaje një matricë dy-dimensionale V-me-V boolean array;

për çdo segment v–w në graf: `adj[v][w] = adj[w][v] = true`.



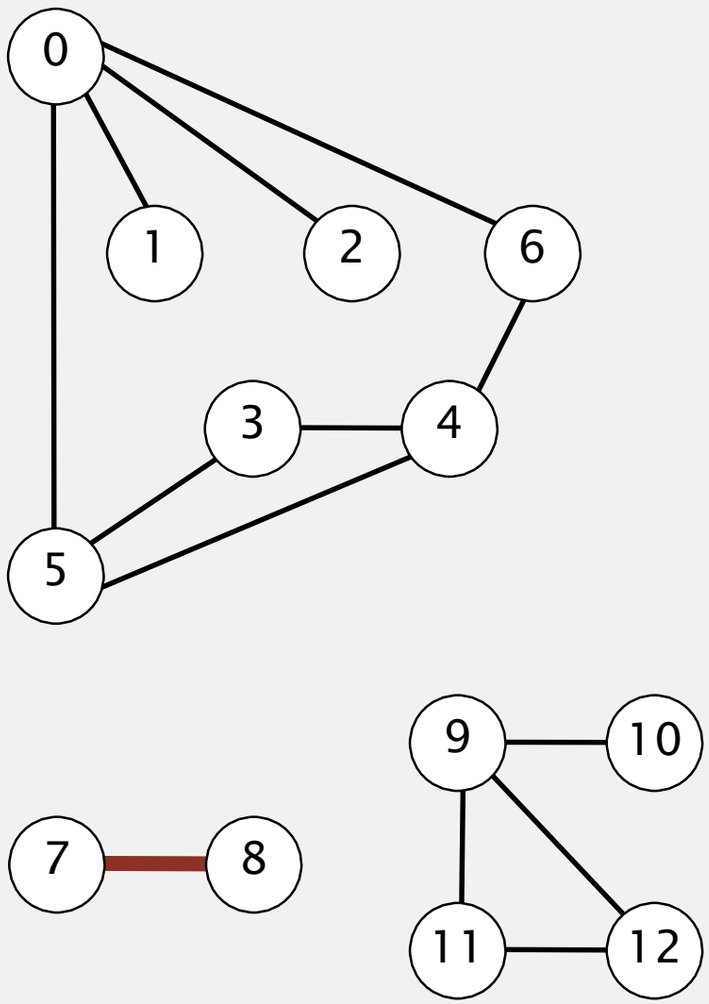
two entries for each edge

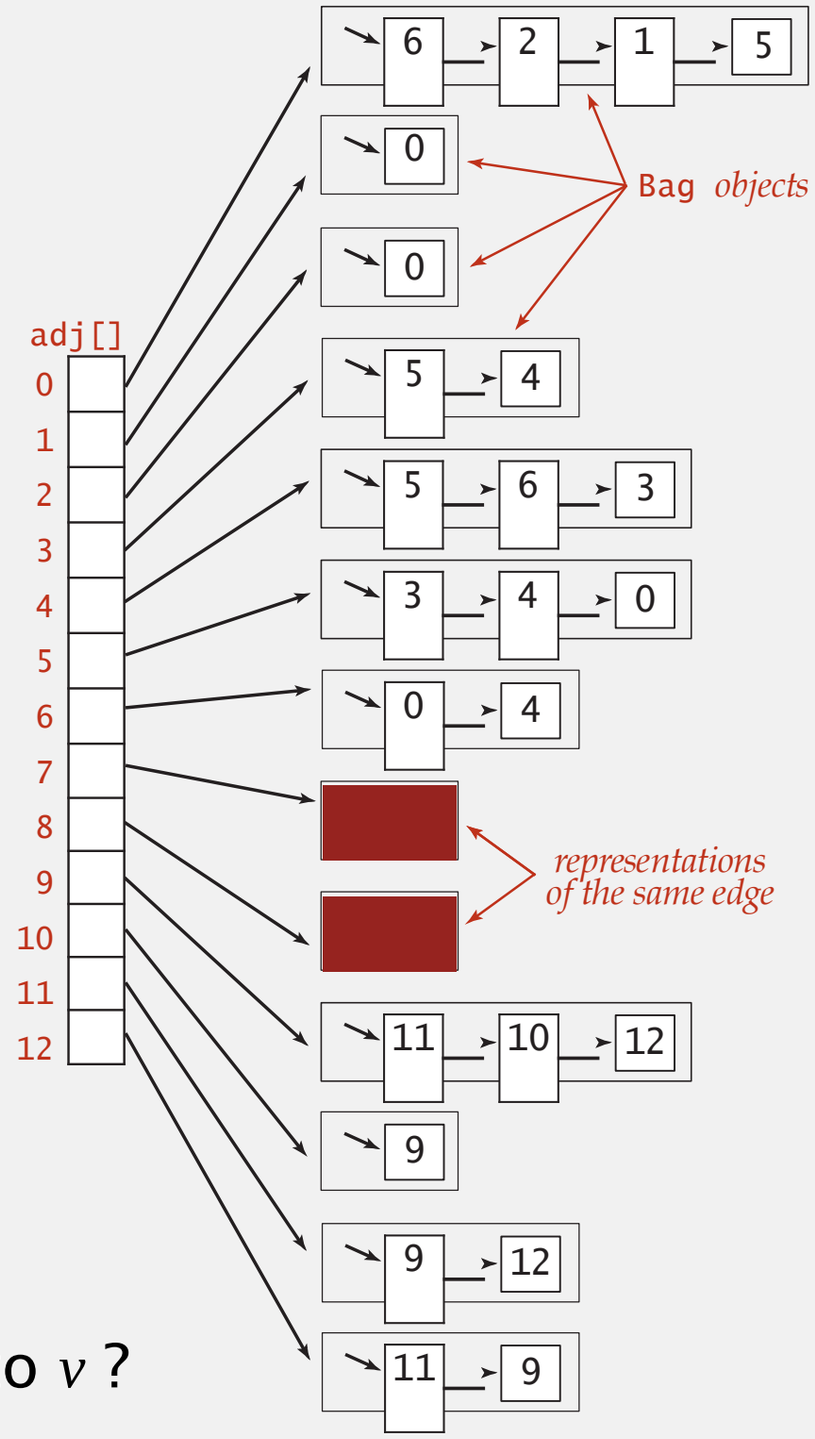
	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0

Q. How long to iterate over vertices adjacent to v ?

Paraqitja me Adjacency-list graf

Mirëmbaje vertex-indexed array of lists.





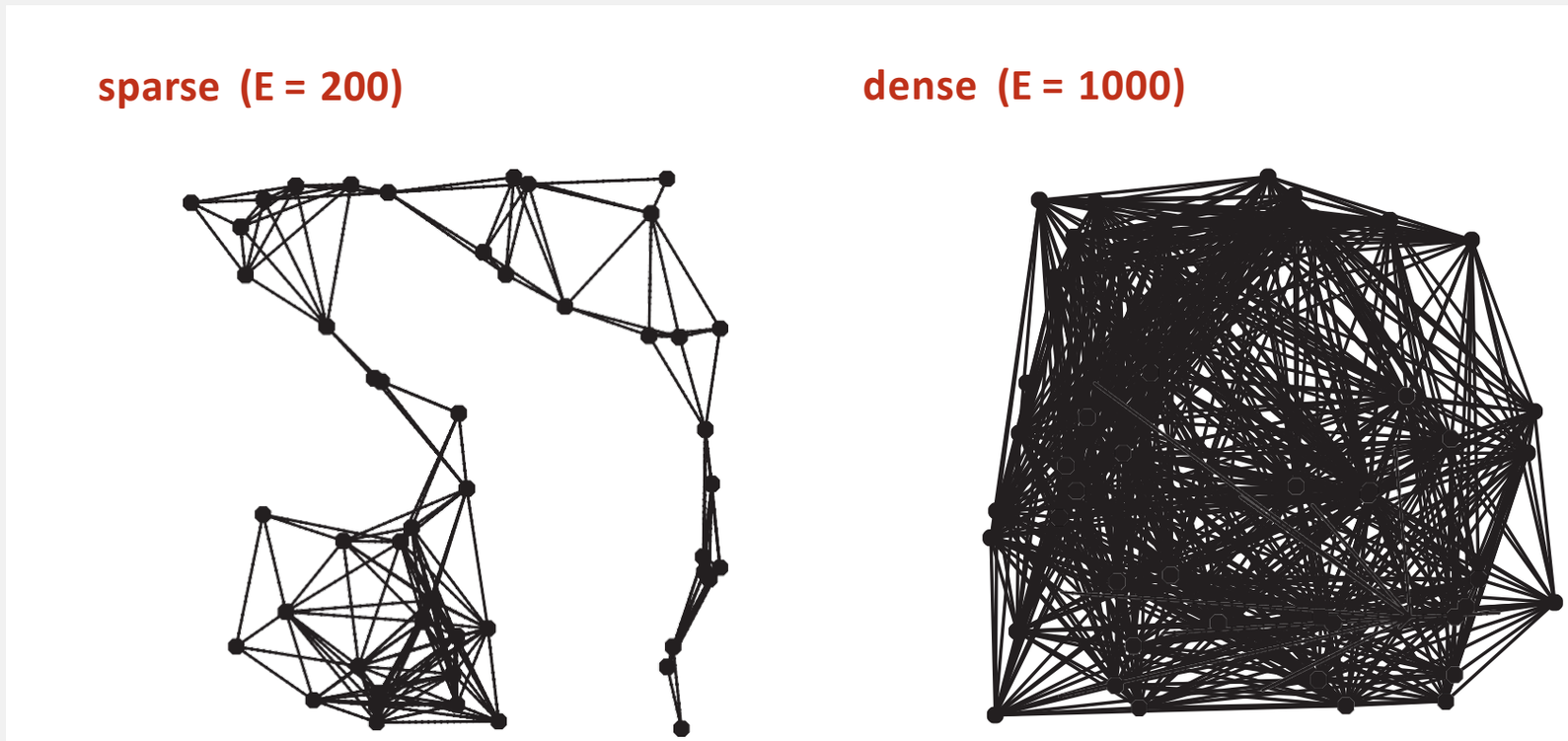
Q. How long to iterate over vertices adjacent to v ?

Paraqitja e grafeve

Në praktikë. Të përdoret paraqitja me adjacency-lists.

- Algoritme që bazohen në iterim nëpër kulmet e ngjitura me v.
- Grafet e vërteta kanë tendencë të jenë sparse (të rralla).

huge number of vertices, small average vertex degree



Two graphs (V = 50)

Graph representations

In practice. Use adjacency-lists representation.

- Algorithms based on iterating over vertices adjacent to v.
- Real-world graphs tend to be sparse.

huge number of vertices,
small average vertex degree

representation	space	add edge	edge between v and w?	iterate over vertices adjacent to v?
list of edges	E	1	E	E
adjacency matrix	V^2	1 *	1	V
adjacency lists	$E + V$	1	$degree(v)$	$degree(v)$

* disallows parallel edges

Paraqitja e grafit me adjacency-list: Java implementation

```
public class Graph
{
   private final int V;
   private Bag<Integer>[] adj;

   public Graph(int V)
   {
      this.V = V;
      adj = (Bag<Integer>[]) new Bag[V];
      for (int v = 0; v < V; v++)
         adj[v] = new Bag<Integer>();
   }

   public void addEdge(int v, int w)
   {
      adj[v].add(w);
      adj[w].add(v);
   }

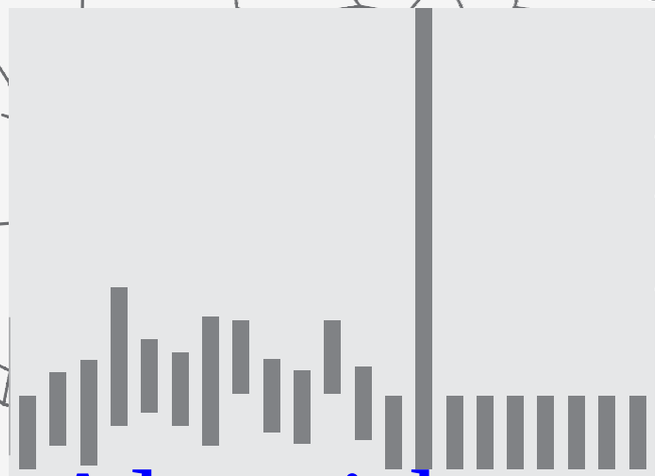
   public Iterable<Integer> adj(int v)
   {  return adj[v];  }
}
```

adjacency lists
(using Bag data type)

create empty graph
with V vertices

add edge v-w
(parallel edges and
self-loops allowed)

iterator for vertices adjacent to v



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

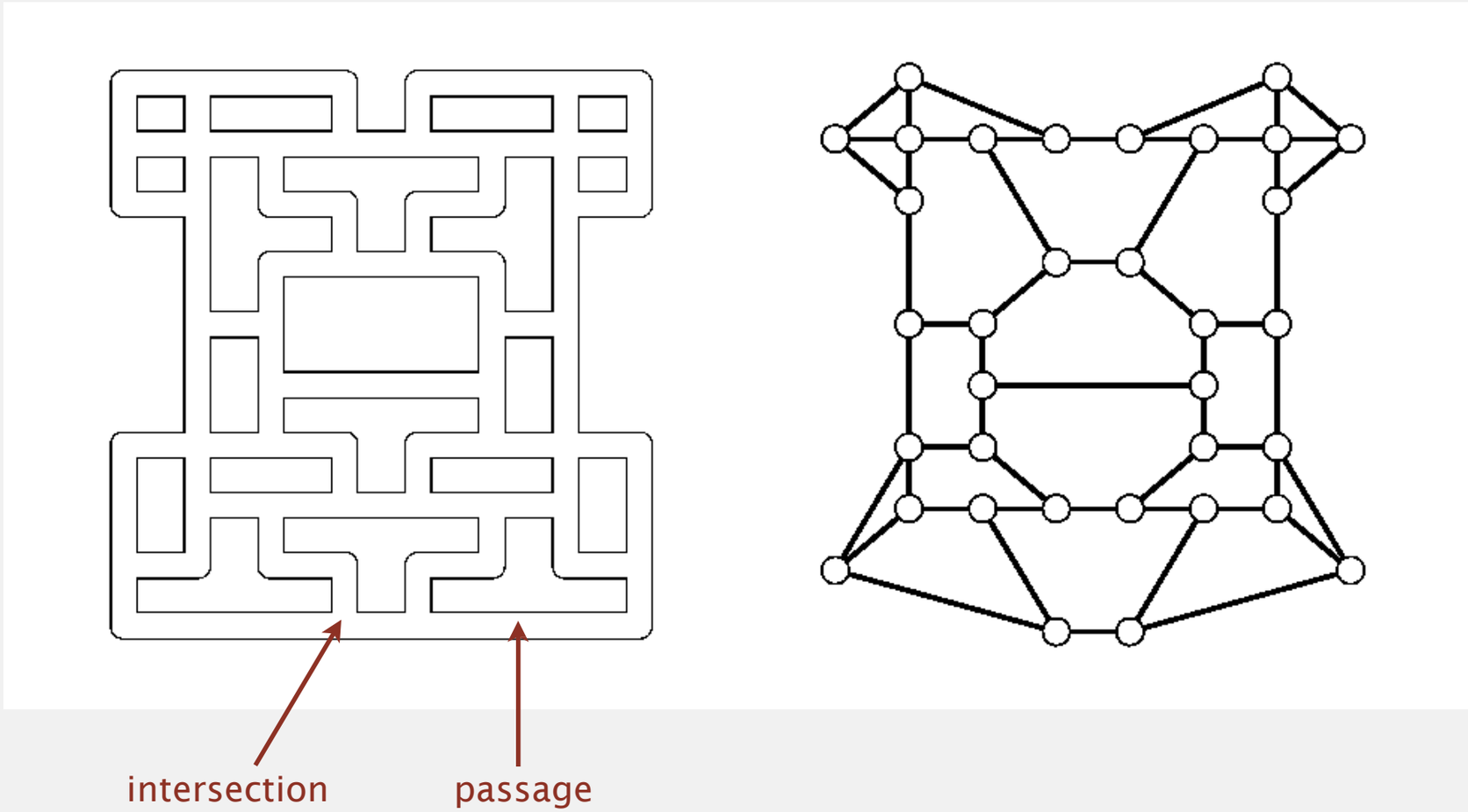
4.1 UNDIRECTED GRAPHS

- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

Hulumtimi i labirintit

Grafi i labirintit.

- Kulmi = udhëkryq.
- Segmenti = rruga.

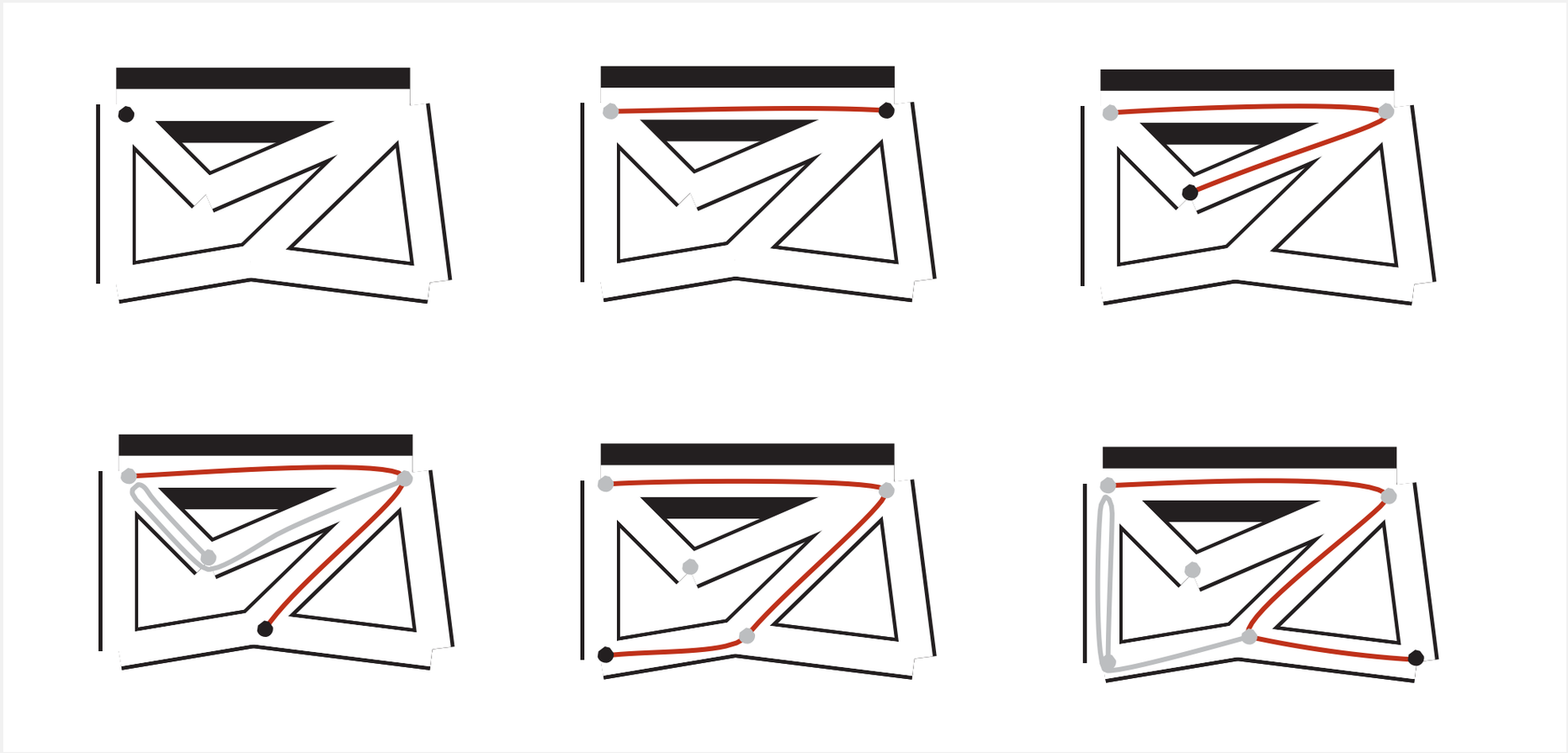


Goal. Të hulumtohet çdo udhëkryq në labirint

Trémaux huluimtimi i labirintit

Algoritmi.

- Shpështjello një lëmsh pas vetit.
- Shëno çdo udhëkryq dhe rrugë të vizituar.
- Kthehu prapa kur shpenzohen të gjitha opcionet.



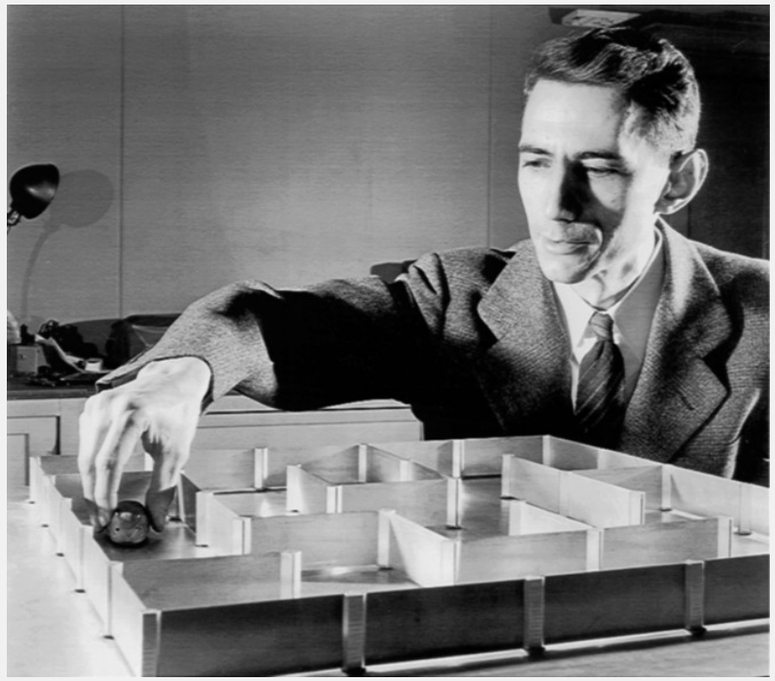
Trémaux maze exploration

Algorithm.

- Shpështjello një lëmsh pas vetit.
- Shëno çdo udhëkryq dhe rrugë të vizituar.
- Kthehu prapa kur shpenzohen të gjitha opcionet.

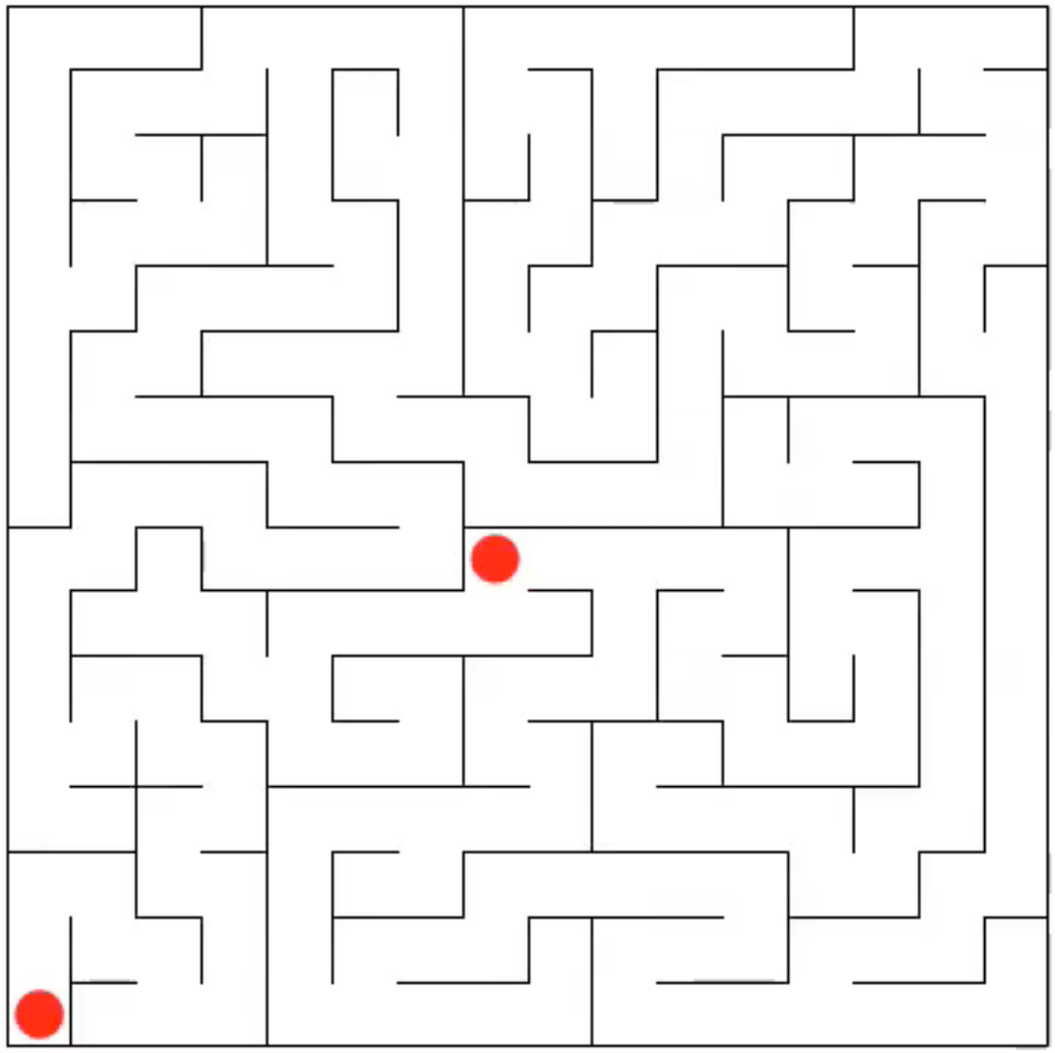


The Labyrinth (with Minotaur)

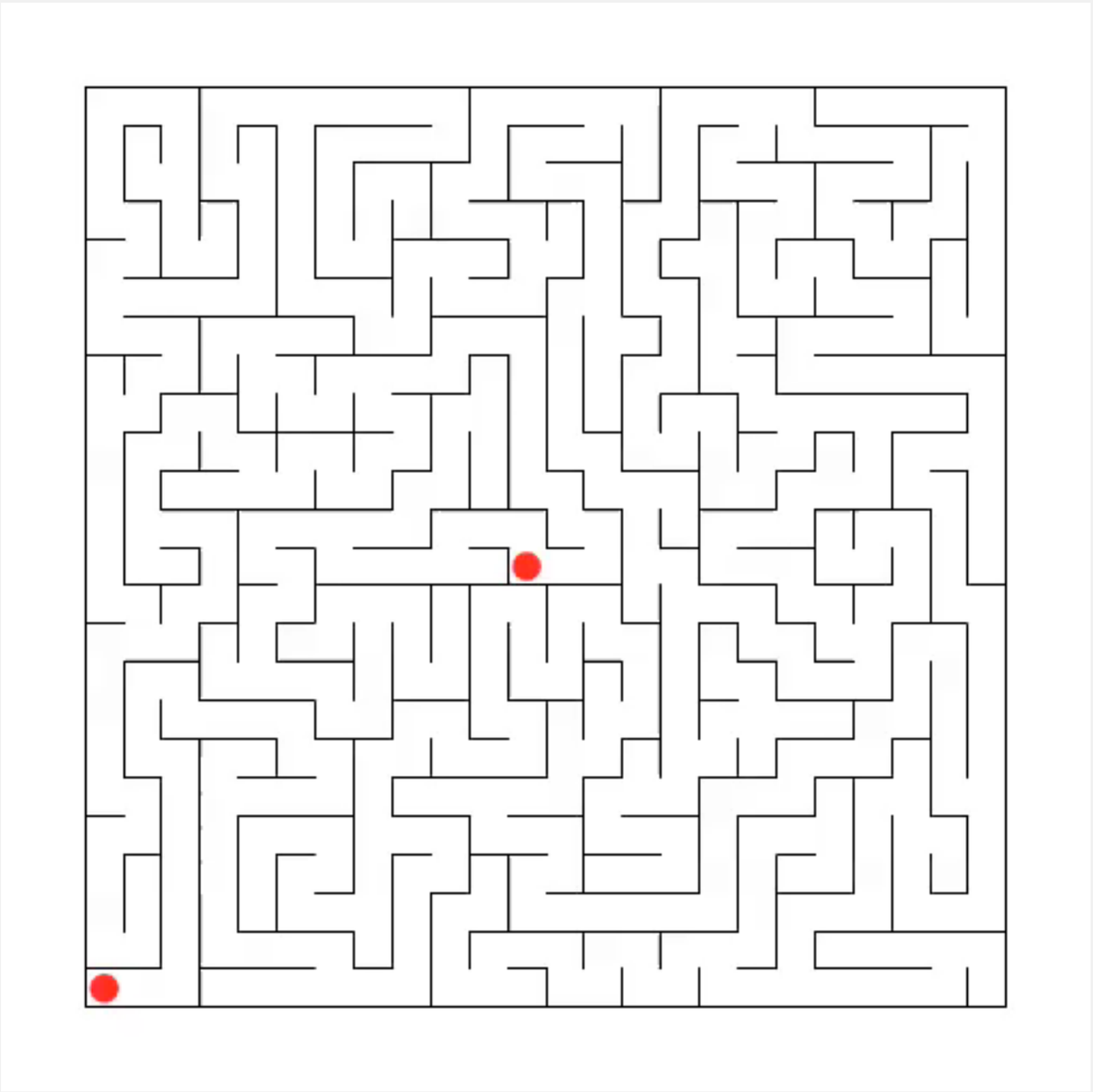


Claude Shannon (with Theseus mouse)

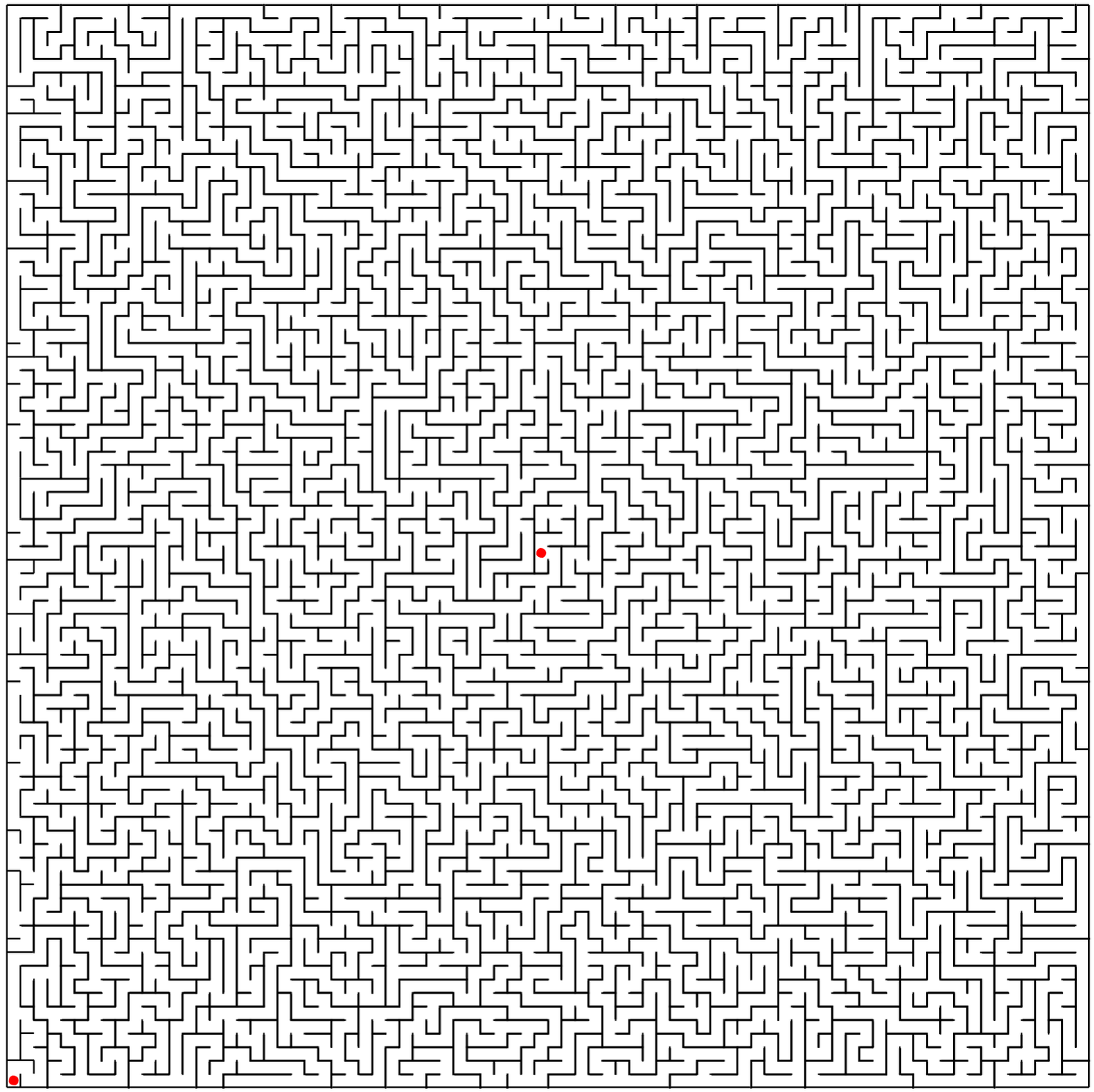
Hulumtimi i labirintit: lehtë



Hulumtimi i labirintit: mesatar



Hulumtimi i labirintit: sfidë për të humbur kohë



Depth-first search

Qëllimi. Përshko grafin në mënyrë sistematike.

Idea. Imito hulumtimin e labirintit. ← function-call stack acts as ball of string

DFS (to visit a vertex v)

Shëno v për të vizituarat.

Vizito të gjitha kulmet e pashënuara w të lidhura me v në mënyrë të vazhdueshme

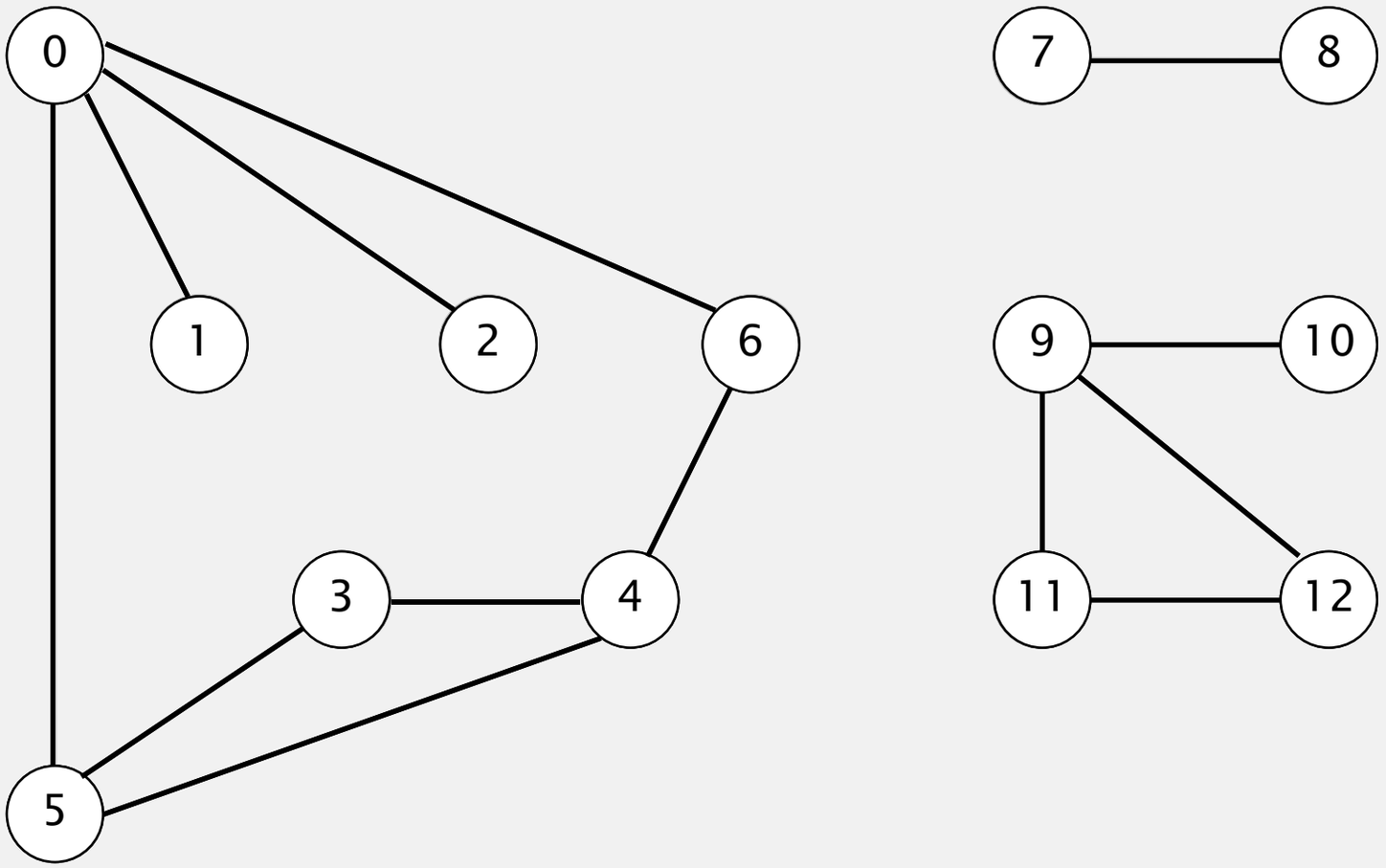
Aplikimet tipike.

- Gjej të gjitha kulmet e lidhura me një kulm burimor.
- Gje një rrugë në mes të dy kulmeve.

Depth-first search demo

Për të vizituar një kulm v :

- Shëno kulmin v pas vizitës.
- Vizito vazhdimisht të gjitha kulmet e pashënuara, fqinje me v.



graph G

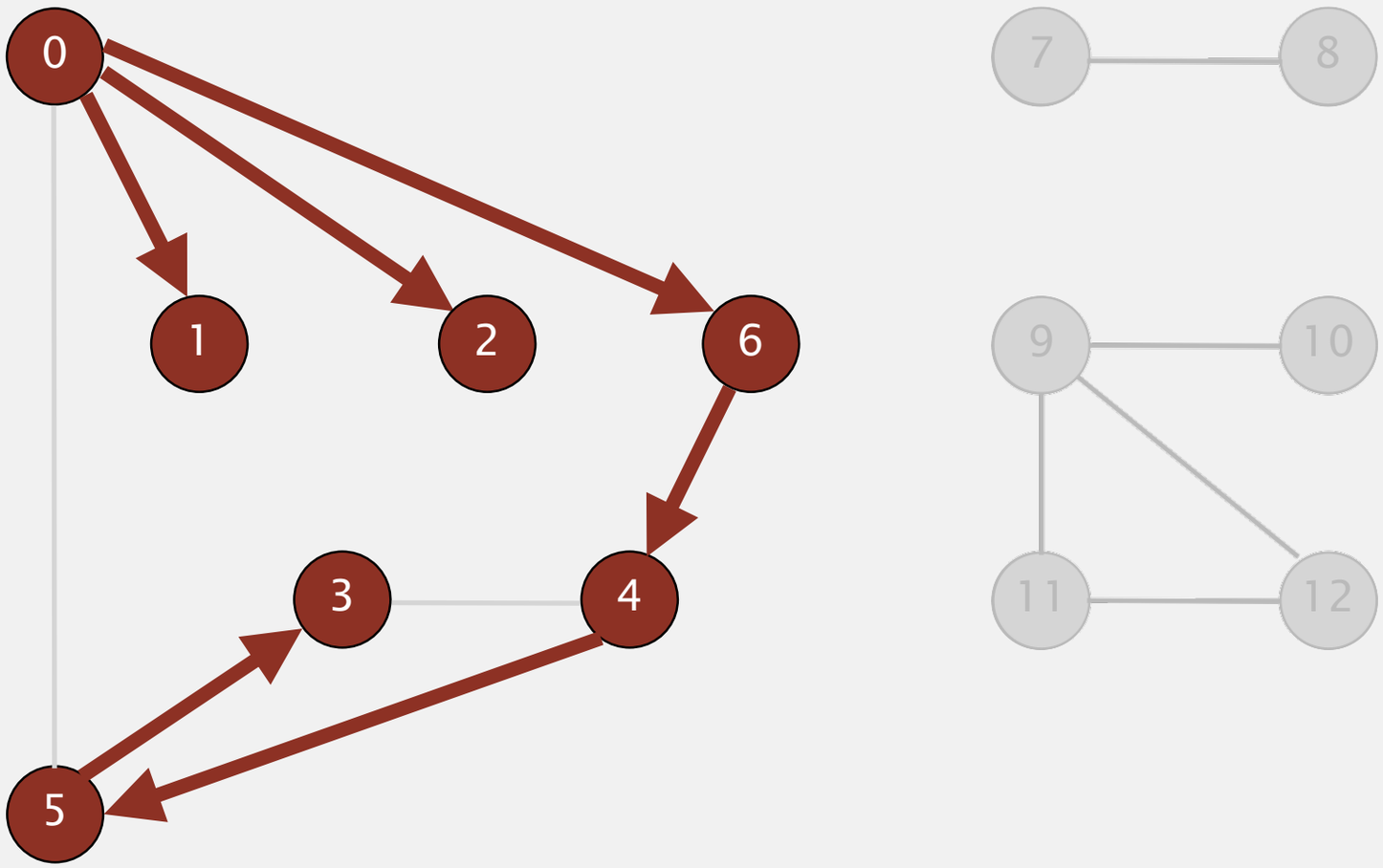
tinyG.txt

```
V → 13
    13   ← E
 0  5
 4  3
 0  1
 9  12
 6  4
 5  4
 0  2
11  12
 9  10
 0  6
 7  8
 9  11
 5  3
```

Depth-first search demo

Përtëvizituar një kulm v :

- Shënoni kulmin v si të vizituar.
- Vazhdimisht vizito të gjitha kulmet e pashënuara të lidhura me v.



v	marked[]	edgeTo[]
0	T	-
1	T	0
2	T	0
3	T	5
4	T	6
5	T	4
6	T	0
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

vertices reachable from 0

Depth-first search: Java implementation

```
public class DepthFirstPaths
{
   private boolean[] marked;
   private int[] edgeTo;
   private int s;

   public DepthFirstPaths(Graph G, int s)
   {
      ...
      dfs(G, s);
   }

   private void dfs(Graph G, int v)
   {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w])
         {
            dfs(G, w);
            edgeTo[w] = v;
         }
   }
}
```

marked[v] = true if v connected to s

edgeTo[v] = previous vertex on path from s to v

initialize data structures

find vertices connected to s

recursive DFS does the work

Depth-first search: vetitë

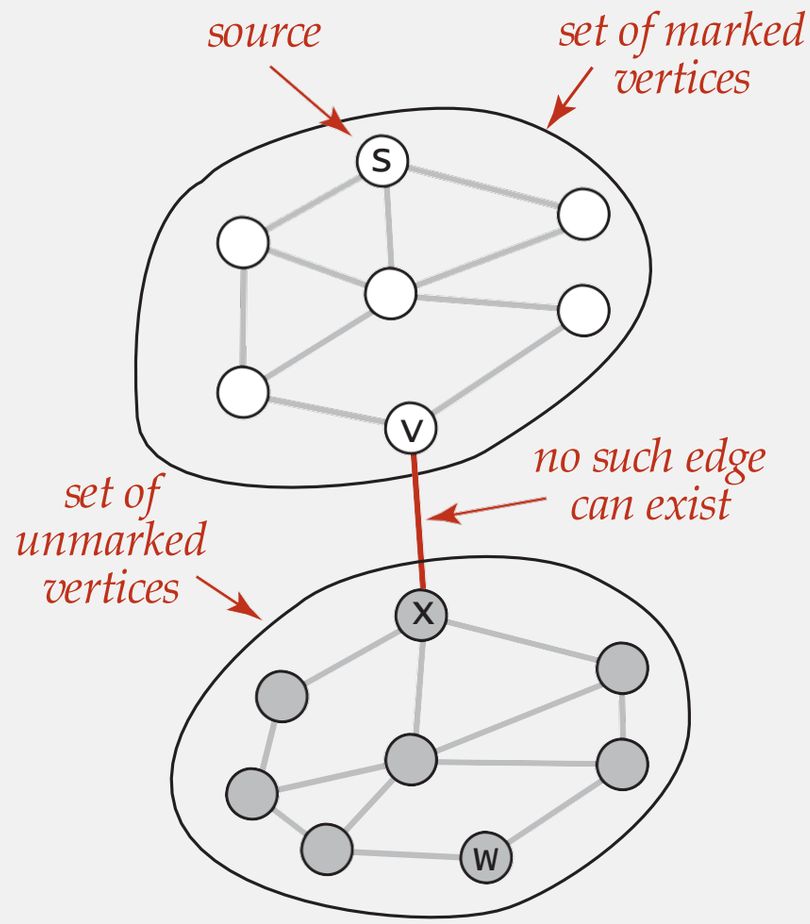
Proposition. DFS marks all vertices connected to s in time proportional to the sum of their degrees (plus time to initialize the `marked[]` array).

Pf. [correctness]

- If w marked, then w connected to s (why?)
- If w connected to s, then w marked.
  (if w unmarked, then consider last edge on a path from s to w that goes from a marked vertex to an unmarked one).

Pf. [running time]

Each vertex connected to s is visited once.



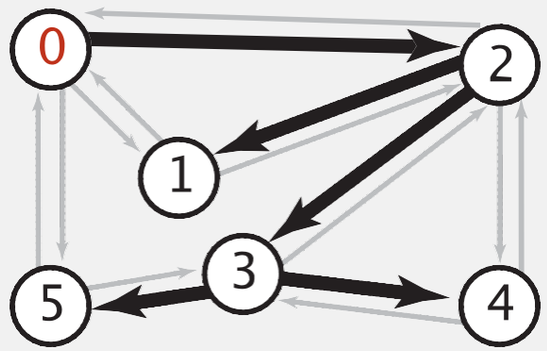
Depth-first search: properties

Proposition. After DFS, can check if vertex v is connected to s in constant time and can find v–s path (if one exists) in time proportional to its length.

Pf. `edgeTo[]` is parent-link representation of a tree rooted at vertex `s`.

```
public boolean hasPathTo(int v)
{  return marked[v];  }

public Iterable<Integer> pathTo(int v)
{
   if (!hasPathTo(v)) return null;
   Stack<Integer> path = new Stack<Integer>();
   for (int x = v; x != s; x = edgeTo[x])
      path.push(x);
   path.push(s);
   return path;
}
```



edgeTo[]

0	
1	2
2	0
3	2
4	3
5	3

Depth-first search application: flood fill

Sfida. Flood fill (Photoshop magic wand).

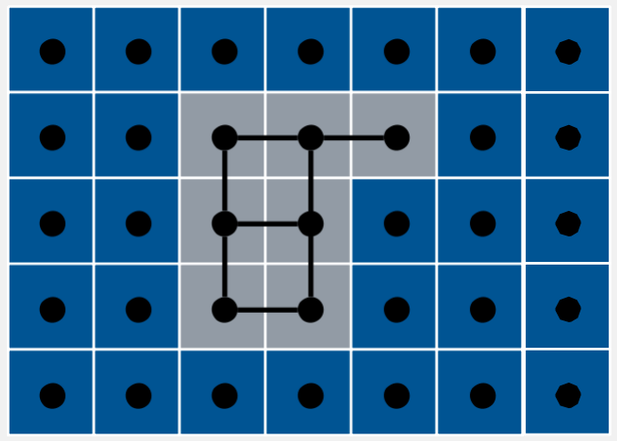
Supozim. Foto ka miliona deri në miliarda of pixela.



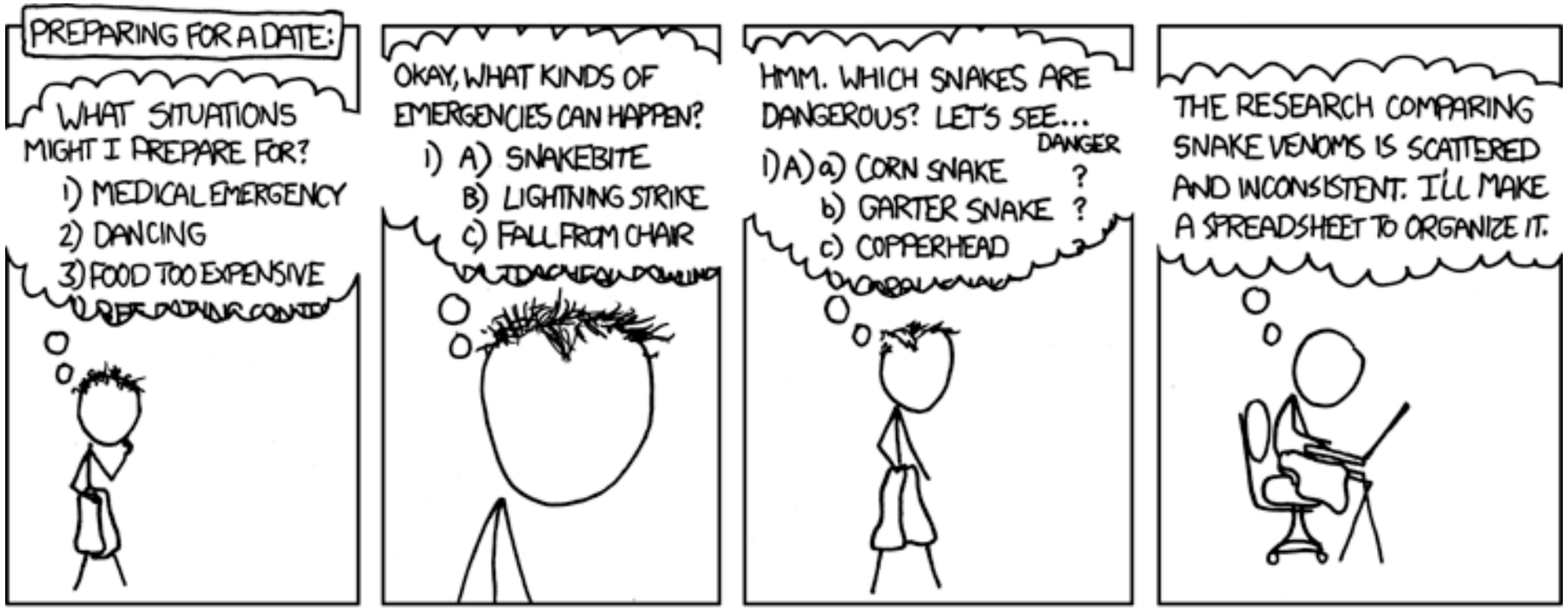


Solution. Build a grid graph (implicitly).

- Kulmi: Pixel
- Edge: lidhjet në të përhimtë



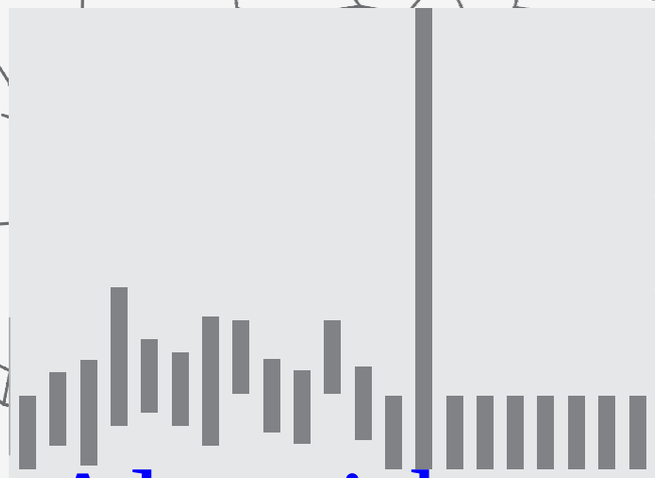
Depth-first search application: preparing for a date





xkcd

http://xkcd.com/761/



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

4.1 UNDIRECTED GRAPHS

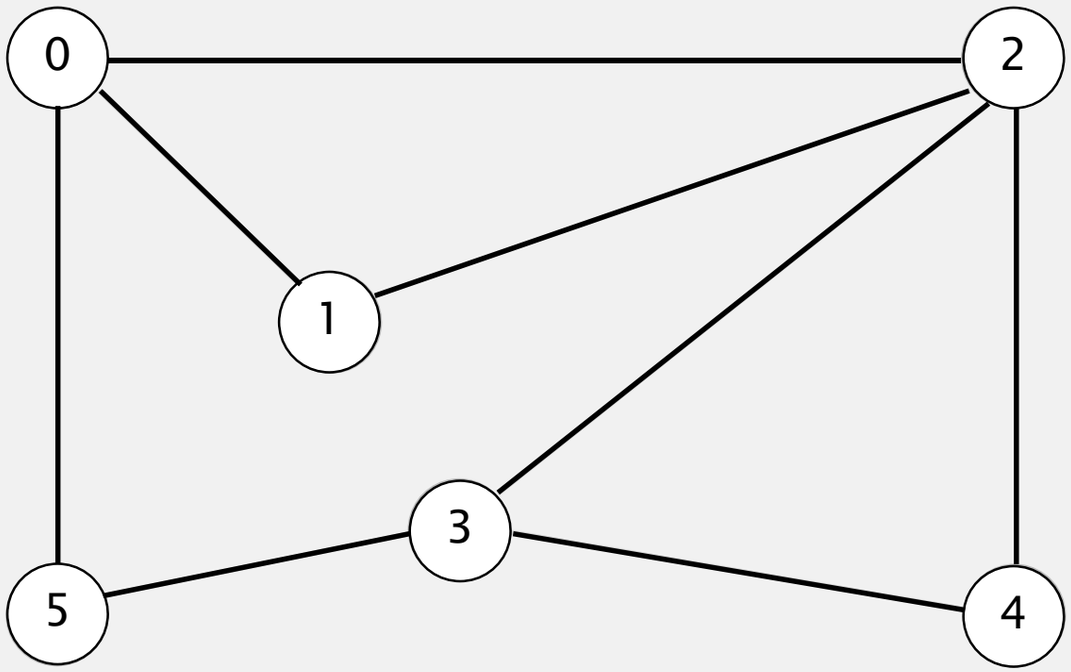
- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

Breadth-first search demo

Përsërit derisa queue is empty:



- Remove kulmin v prej queue.
- Add në queue të gjitha kulmet e lidhura (të pashënuara) me v dhe shënoi.



tinyCG.txt

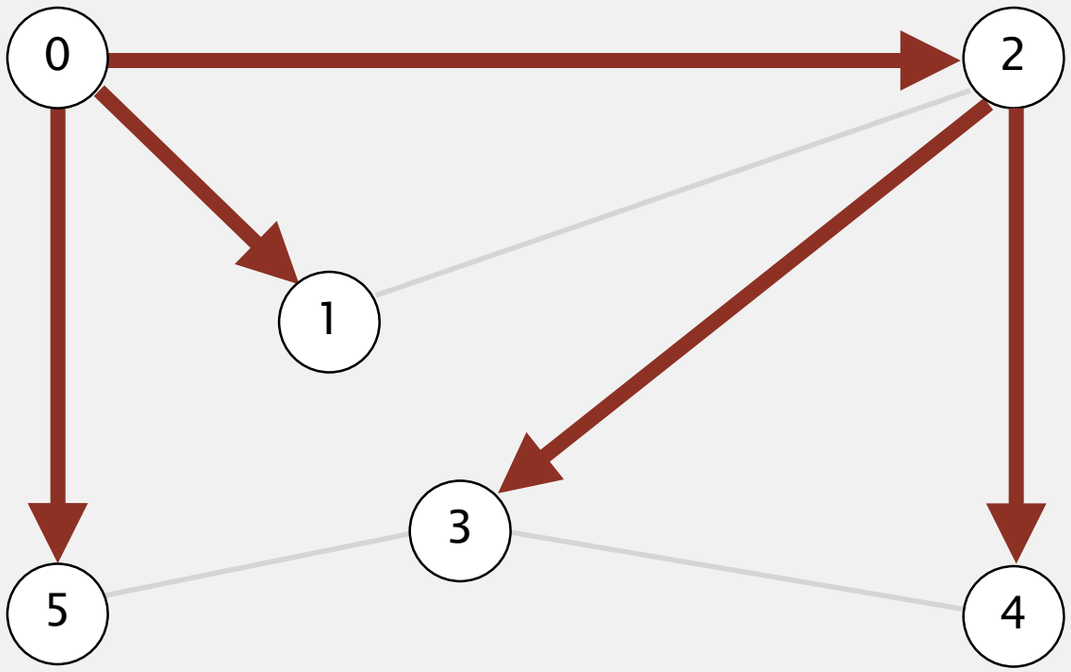
```
V
6
8   E
0 5
2 4
2 3
1 2
0 1
3 4
3 5
0 2
```

graph G

Breadth-first search demo

Përsërit derisa queue is empty:

- Remove kulmin v prej queue.
- Add në queue të gjitha kulmet e lidhura (të pashënuara)me v dhe shënoi.



v	edgeTo[]	distTo[]
0	-	0
1	0	1
2	0	1
3	2	2
4	2	2
5	0	1

done

Breadth-first search

Përsërit derisa queue is empty :

- Remove kulmin v prej queue.
- Add në queue të gjitha kulmet e lidhura (të pashënuara)me v dhe shënoi.

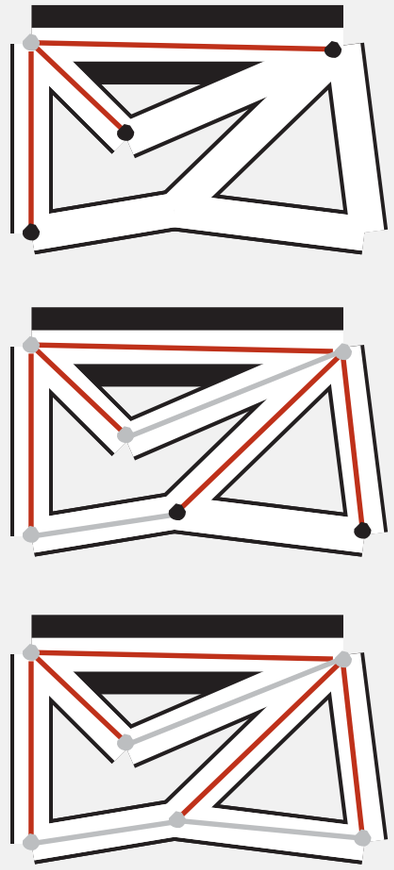
BFS (prej burimit kulmi s)

Vendos s në FIFO queue, dhe shëno s si të vizituar.

Përsërit derisa queue is empty:

- remove kulmin v më të parë added

- add çdo fqinj të pavizituar të v në queue, dhe shënoi si të vizituar.



Breadth-first search: Java implementation

```
public class BreadthFirstPaths
{
   private boolean[] marked;
   private int[] edgeTo;
   private int[] distTo;

   …

   private void bfs(Graph G, int s) {
      Queue<Integer> q = new Queue<Integer>();
      q.enqueue(s);
      marked[s] = true;
      distTo[s] = 0;

      while (!q.isEmpty()) {
         int v = q.dequeue();
         for (int w : G.adj(v)) {
            if (!marked[w]) {
               q.enqueue(w);
               marked[w] = true;
               edgeTo[w] = v;
               distTo[w] = distTo[v] + 1;
            }
         }
      }
   }
}
```

initialize FIFO queue of vertices to explore

found new vertex w via edge v-w

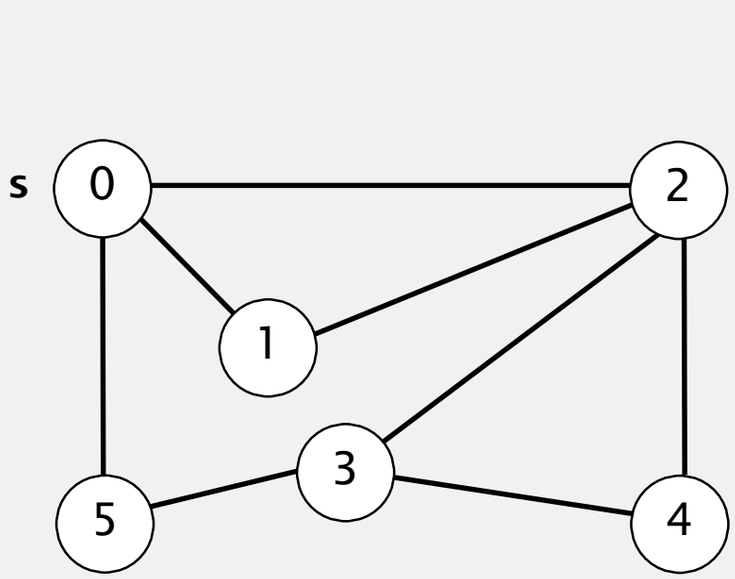
Breadth-first search properties

Q. In which order does BFS examine vertices?

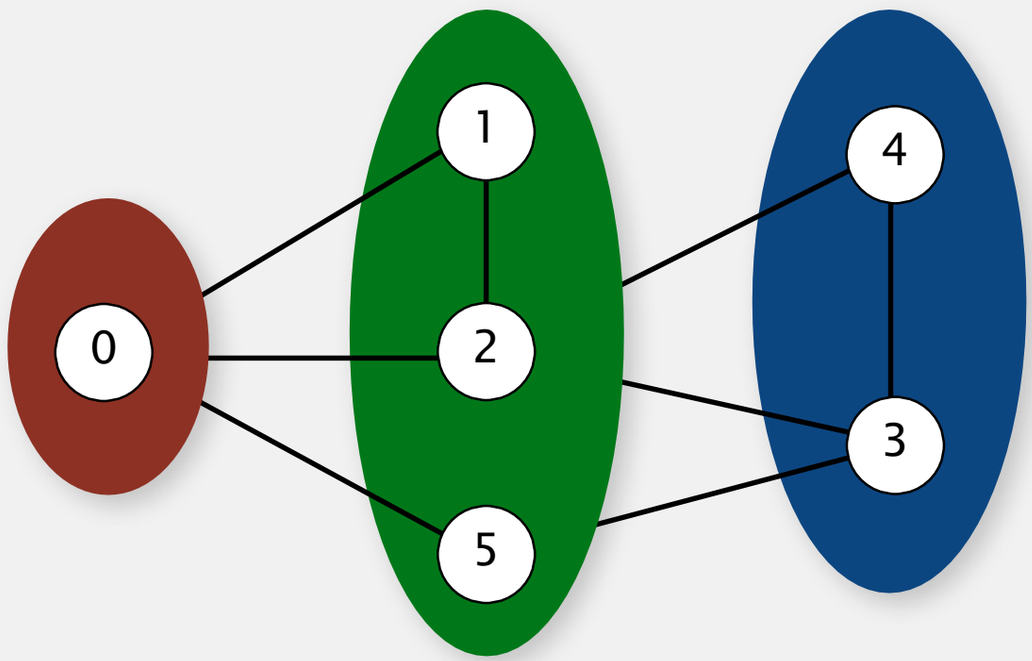
A. Increasing distance (number of edges) from s.

queue always consists of ≥ 0 vertices of distance k from s,
followed by ≥ 0 vertices of distance k+1

Proposition. In any connected graph G, BFS computes shortest paths from s to all other vertices in time proportional to $E + V$.



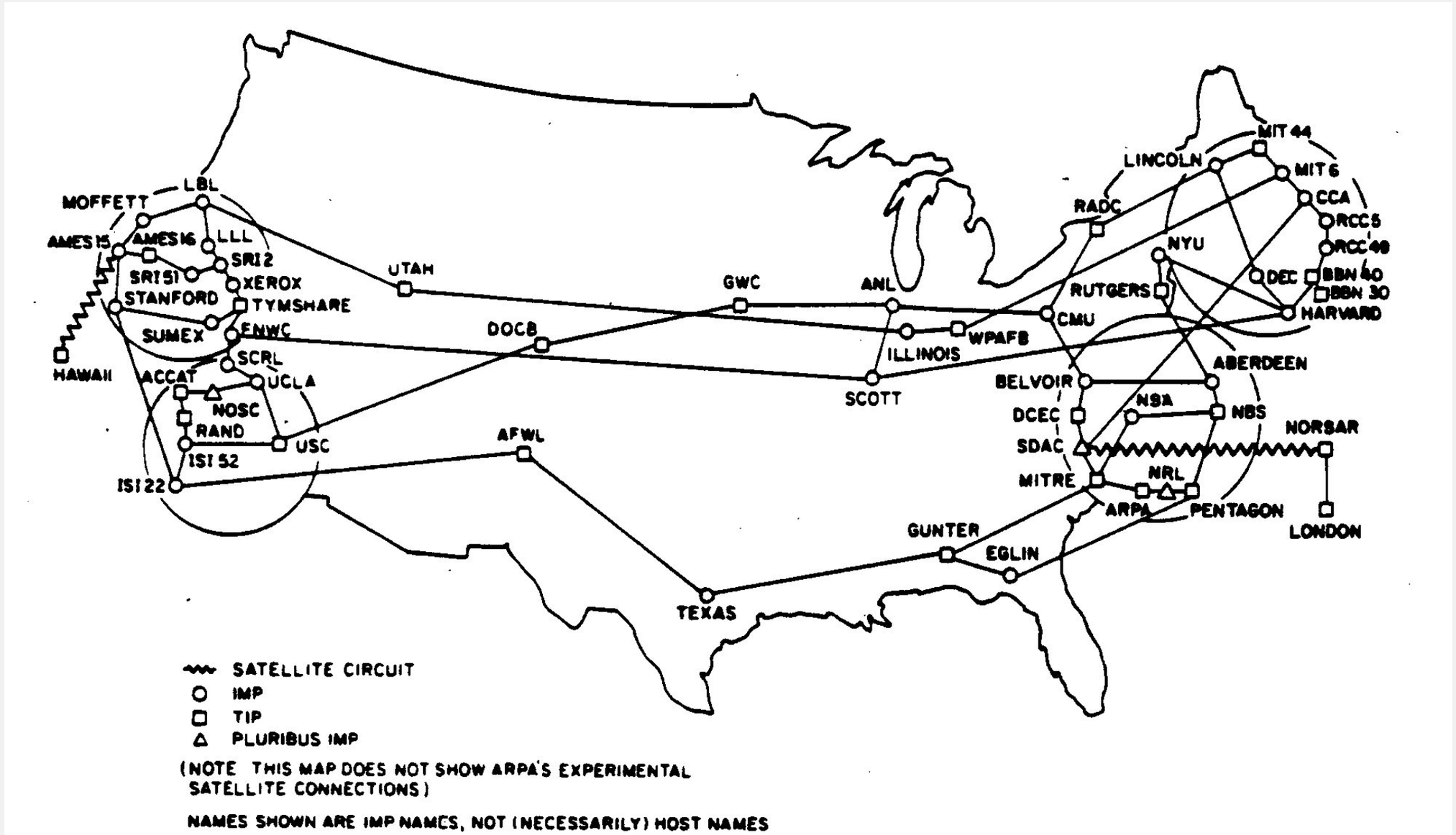
graph G



dist = 0 dist = 1 dist = 2

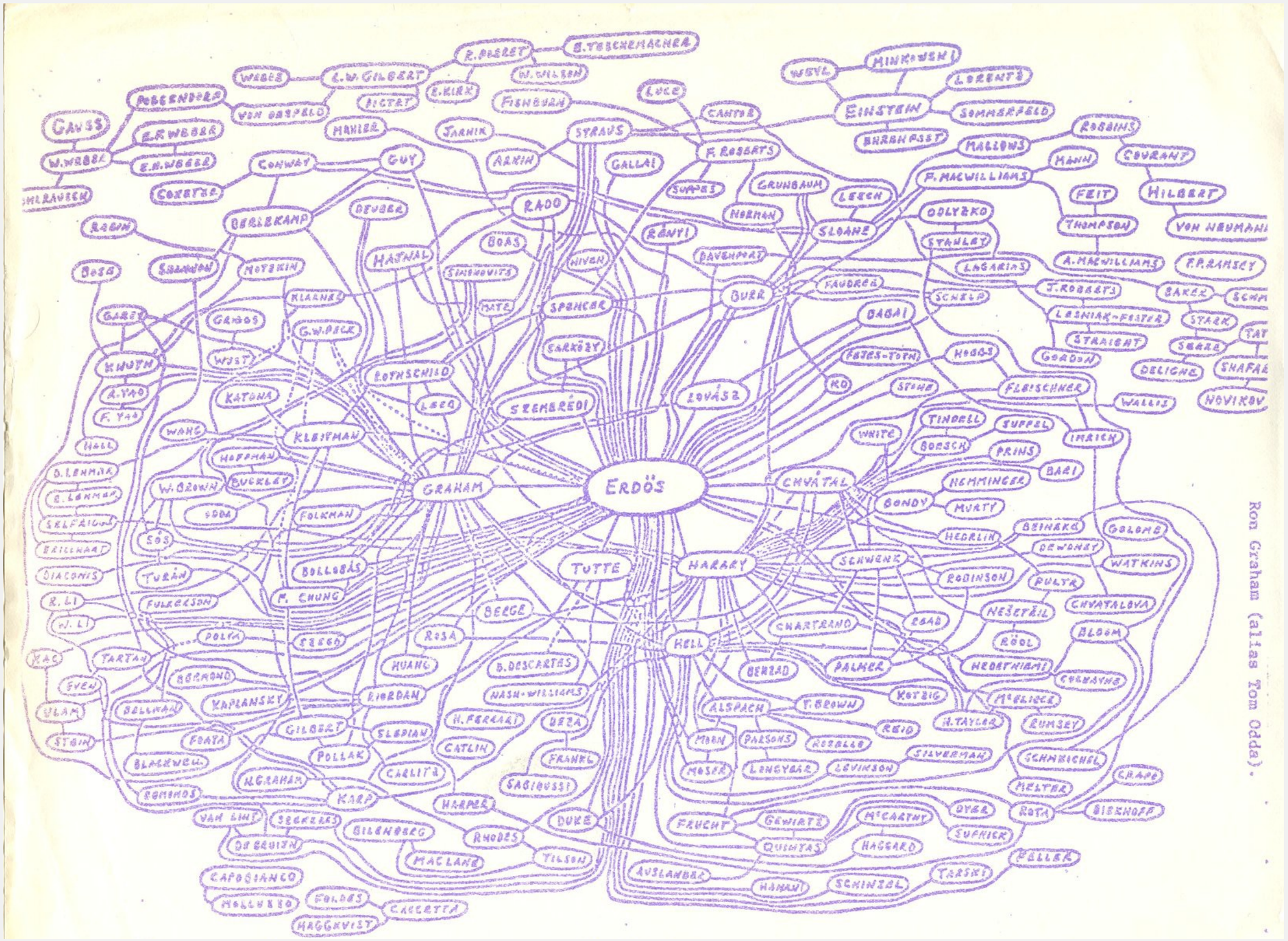
Breadth-first search application: routing

Fewest number of hops in a communication network.

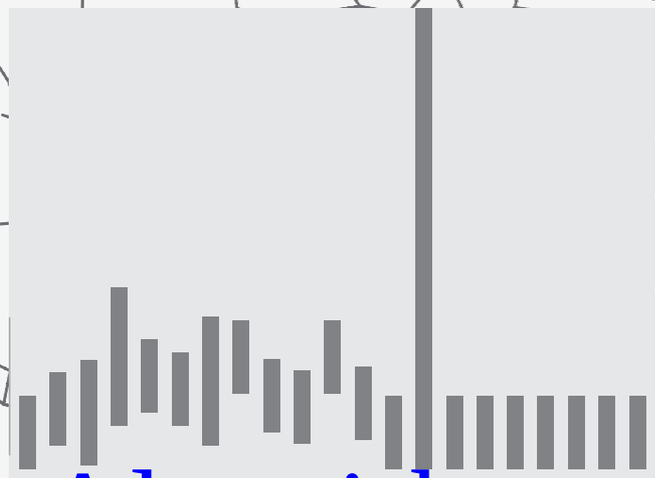


ARPANET, July 1977

Breadth-first search application: Erdös numbers



hand- drawing of part of the Erdös graph by Ron Graham



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

4.1 UNDIRECTED GRAPHS

- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

Connectivity queries

Def. Kulmet v dhe w janë connected ekziston një rrugë në mes të tyre.

Qëllimi. Preprocess graph to answer queries of the form *is v connected to w?* in constant time.

public class CC		
	CC(Graph G)	*find connected components in G*
boolean	connected(int v, int w)	*are v and w connected?*
int	count()	*number of connected components*
int	id(int v)	*component identifier for v* (*between* `0` *and* `count() - 1`)

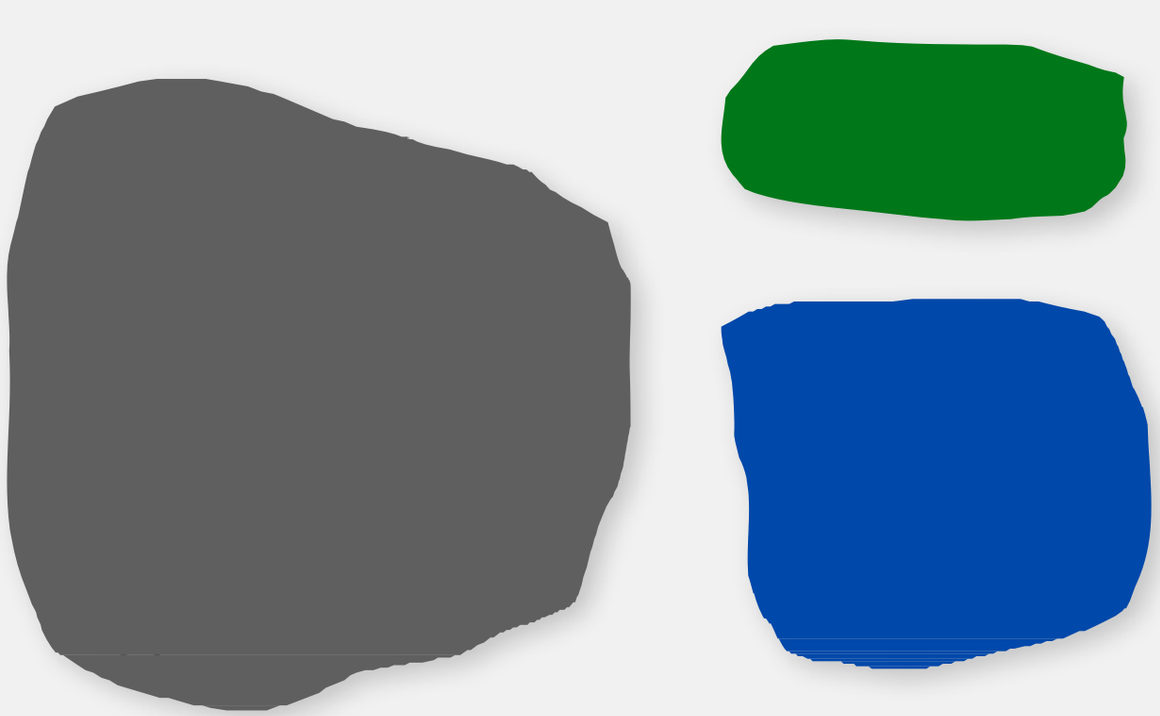
.

Connected components

The relation "is connected to" is an equivalence relation:

- Reflexive: v is connected to v.
- Symmetric: if v is connected to w, then w is connected to v.
- Transitive: if v connected to w and w connected to x, then v connected to x.

Def. A connected component is a maximal set of connected vertices.



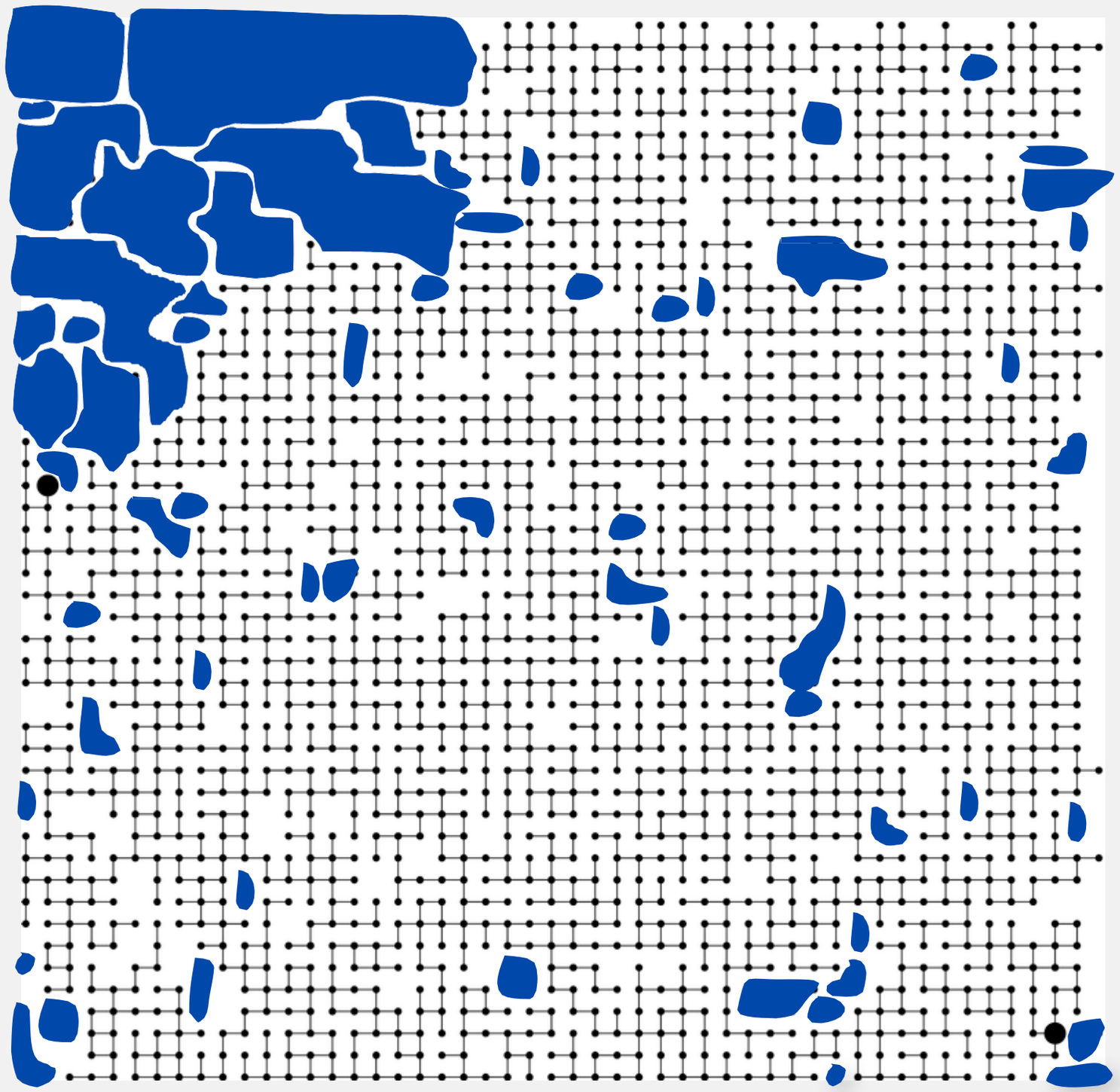
3 connected components

v	id[]
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	1
9	2
10	2
11	2
12	2

Remark. Given connected components, can answer queries in constant time.

Connected components

Def. A connected component është një bashkësi maksimale e kulmeve të lidhura.



63 connected components

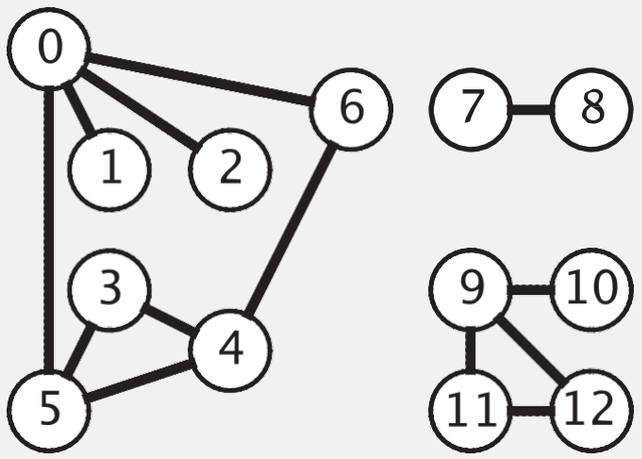
Connected components

Qëllimi. Ndajë kulmet në connected components.

Connected components

Inicializo të gjitha kulmet v si të pashënuara.

Për çdo kulm v të pashënuar, DFS për të identifikuar të gjitha kulmet e zbuluara si pjesë e component të njejtë.



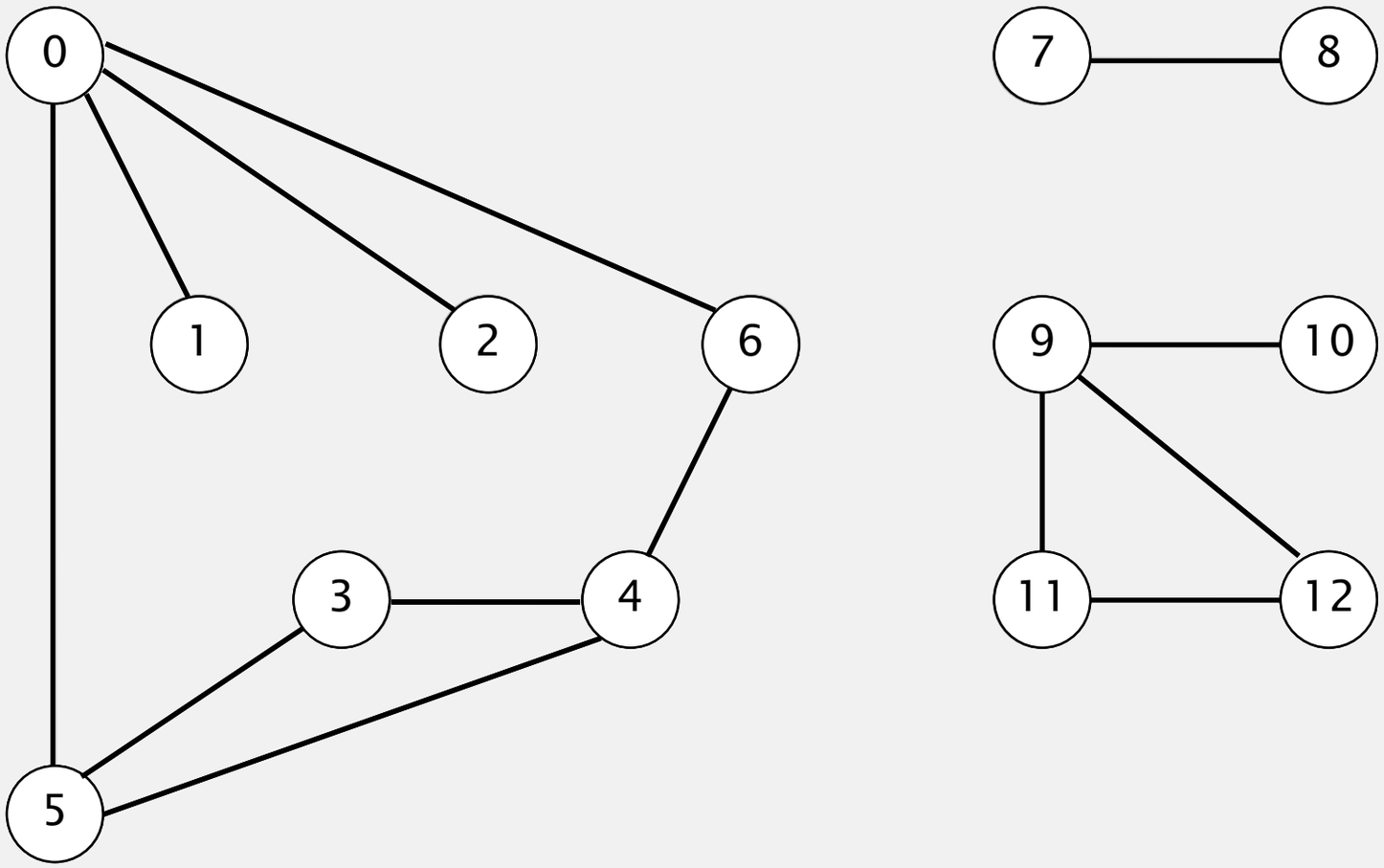
tinyG.txt

```
V → 13
13 ← E
0  5
4  3
0  1
9  12
6  4
5  4
0  2
11  12
9  10
0  6
7  8
9  11
5  3
```

Connected components demo

Përtëvizituar një kulm v :

- Shëno kulmin v si i vizituar.
- Vazhdimisht vizito të gjitha kulmet e pashënuara të lidhura me v.



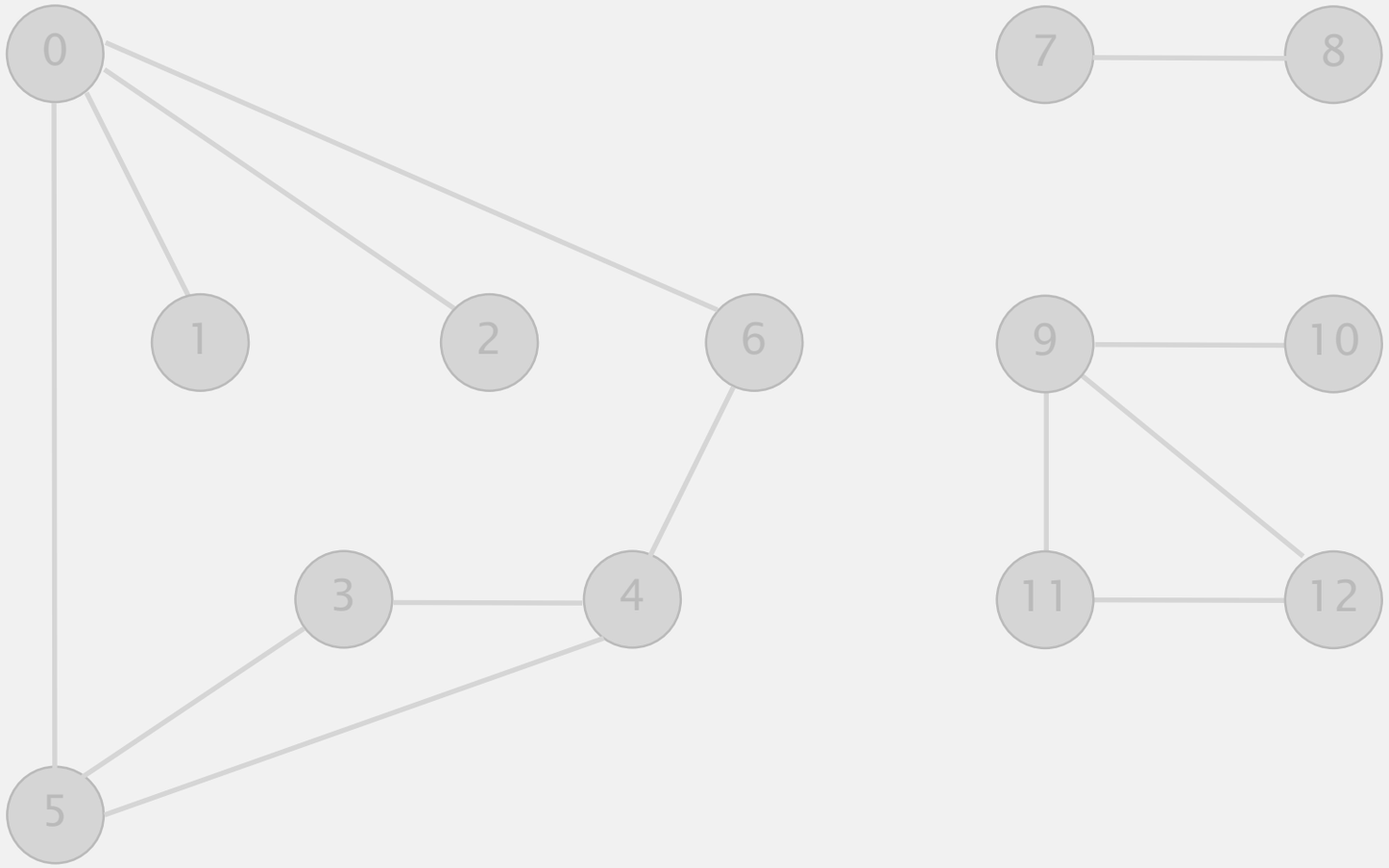
v	marked[]	id[]
0	F	-
1	F	-
2	F	-
3	F	-
4	F	-
5	F	-
6	F	-
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

graph G

Connected components demo

To visit a vertex v :

- Shëno kulmin v si i vizituar.
- Vazhdimisht vizito të gjitha kulmet e pashënuara të lidhura me v.



v	marked[]	id[]
0	T	0
1	T	0
2	T	0
3	T	0
4	T	0
5	T	0
6	T	0
7	T	1
8	T	1
9	T	2
10	T	2
11	T	2
12	T	2

done

Finding connected components with DFS

```
public class CC
{
   private boolean[] marked;
   private int[] id;
   private int count;

   public CC(Graph G)
   {
      marked = new boolean[G.V()];
      id = new int[G.V()];
      for (int v = 0; v < G.V(); v++)
      {
         if (!marked[v])
         {
            dfs(G, v);
            count++;
         }
      }
   }

   public int count()
   public int id(int v)
   public boolean connected(int v, int w)
   private void dfs(Graph G, int v)
}
```

id[v] = id of component containing v

number of components

run DFS from one vertex in each component

see next slide

Finding connected components with DFS (continued)

```
public int count()
{  return count;  }
```

number of components

```
public int id(int v)
{  return id[v];  }
```

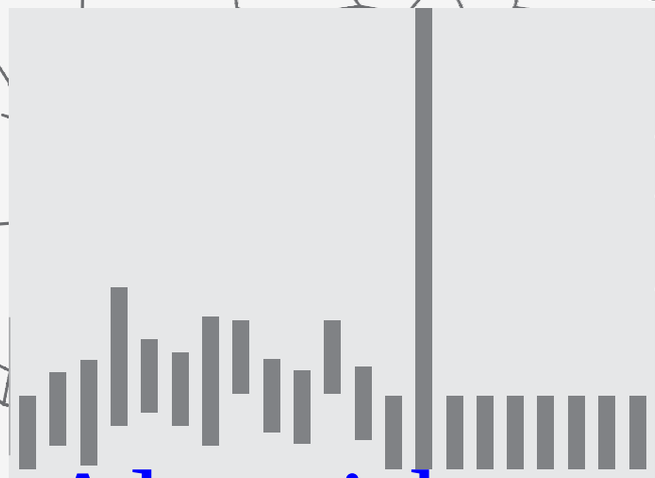
id of component containing v

```
public boolean connected(int v, int w)
{ return id[v] == id[w];  }
```

v and w connected iff same id

```
private void dfs(Graph G, int v)
{
   marked[v] = true;
   id[v] = count;
   for (int w : G.adj(v))
      if (!marked[w])
         dfs(G, w);
}
```

all vertices discovered in same call of dfs have same id



Algorithms

ROBERT SEDGEWICK | KEVIN WAYNE

http://algs4.cs.princeton.edu

4.1 UNDIRECTED GRAPHS

- *introduction*
- *graph API*
- *depth-first search*
- *breadth-first search*
- *connected components*
- *challenges*

Graph traversal summary

BFS and DFS enables efficient solution of many (but not all) graph problems.

problem	BFS	DFS	time
path between s and t	✓	✓	$E + V$
shortest path between s and t	✓		$E + V$
connected components	✓	✓	$E + V$
biconnected components		✓	$E + V$
cycle	✓	✓	$E + V$
Euler cycle		✓	$E + V$
Hamilton cycle			$2^{1.657\,V}$
bipartiteness	✓	✓	$E + V$
planarity		✓	$E + V$
graph isomorphism			$2^{c\,\sqrt{V \log V}}$